WSMA LAB PROJECT · CODE WALKTHROUGH AND VIVA GUIDE

Partial-Input Baselines Reveal Construction Artifacts in Fake News Benchmarks

Every script explained line by line, every abbreviation spelled out, the presenter brief and the questions an examiner is likely to ask.

Generated from the code and results in the repository. Numbers are from the fixed test split (seed 42) unless a mean ± SD is given.

Contents

1. How to use this guide
2. The project in two minutes
3. Glossary: abbreviations and terms
4. Core ideas you need before reading the code
5. How the files fit together
6. Worked example: one real and one fake article through every view
7. Line-by-line walkthrough
    1. run.bat and run_all.py — the one command
    2. scripts/common.py — the shared experiment harness
    3. scripts/transforms.py — the restricted views of an article
    4. scripts/00_get_data.py — fetch and verify data
    5. scripts/01_baseline.py — E1
    6. scripts/02_partial_input.py — E2a, E2b, E2c
    7. scripts/03_stripped.py — E3, E3b
    8. scripts/04_cross_corpus.py — E4, E4b
    9. scripts/05_more_probes.py — E2d, E2e, E3c, SVM check
    10. scripts/06_confounds.py — E5
    11. scripts/07_figures.py — figures and results table
8. Results: what every number means
9. Presenter brief
10. Viva questions and answers

1. How to use this guide

This guide is for the two people defending the project. It explains what every line of the experiment code does and, more importantly, **why** it is written that way, because "why" is what an examiner asks.

Suggested reading order:

- **Everyone:** sections 2, 3 and 4 (the story, the vocabulary, the core ideas), then section 8 (results) and section 10 (questions).
- **Whoever answers code questions:** all of section 7. Sections 7.2 (`common.py`) and 7.3 (`transforms.py`) matter most: every experiment goes through them.
- **Whoever presents:** section 9 is your script. Section 6 gives you concrete examples to point at.

Code listings are copied from the files when this PDF is built, with the real line numbers, so "line 78 of common.py" in this guide is line 78 in the editor. Blank lines are left out.

2. The project in two minutes

The one-line story: *we built a fake news detector, got 98%, then proved the 98% is fake.*

The dataset. ISOT is the most widely used fake news benchmark. Its 44,898 articles come in two files: `True.csv` (21,417 articles, all from the Reuters news agency) and `Fake.csv` (23,481 articles from websites flagged as unreliable). Published classifiers on it report 98–99% accuracy.

The problem. Every real article comes from one newsroom and every fake one from a different set of websites. A model can therefore get the label right by recognising **who wrote the article** (Reuters' house style) without ever judging **whether it is true**. Analogy: if every cancer scan came from Hospital A and every healthy scan from Hospital B, a model can reach 99% by recognising the hospital's scanner, and it fails at Hospital C.

The method. *Partial-input baselines* (Gururangan et al., 2018): train models that are deliberately denied the information they would need to do the task properly, such as the article's content words. If such a crippled model still scores far above chance, the dataset is leaking the answer through something other than the content.

What we found:

- Our detector (E1, full text): **98.4%**. Chance (always guessing the bigger class) is 54.8%.
- First sentence only (E2a): **99.1%**, higher than the full article.
- Only grammatical "function words" and punctuation, no nouns, verbs or names (E2b): **96.8%**.
- Eight numbers such as word count and question marks, no words at all (E2c): **86.2%**.
- Every source marker stripped from both classes (E3b): still **97.3%**.
- US politics only, so topic cannot help, fully stripped, function words only (E5c): **90.3%** against 60.9% chance.
- Tested on other datasets: **45.5%** on LIAR (below its 55.8% chance) and **61.2%** on McIntire's full articles (50.7% chance).

Conclusion. The 98% measures how well a model recognises the publisher's writing style, not how well it detects false news. ISOT cannot tell the two apart, so accuracy on ISOT says little about fake news detection.

How the experiments map to the research questions in the report:

Research question	Experiments	Answer
RQ1 How much accuracy remains when the model is denied the content?	E2a–E2e	Almost all of it: 96.8% from function words alone.
RQ2 How much is lost when source markers are removed (Reuters' markers, then both classes')?	E3 (Reuters markers), E3b (both pipelines' markers), E3c	About one point. The signal is in the whole writing style.
RQ3 Does the detector transfer to independently labelled data?	E4 (LIAR), E4b (McIntire)	No: below chance on LIAR, 61% on McIntire.
RQ4 Is it style, or topic, or the classifier?	E5a–E5d, SVM check	Style. It survives topic matching, and a different classifier does slightly better, not worse.

3. Glossary: abbreviations and terms

3.1 Abbreviations

Short form	In full	What it means here
WSMA	Web and Social Media Analytics	The course this lab project is for.
ISOT	Information Security and Object Technology	The research lab at the University of Victoria (Canada) that built the dataset; the dataset is named after it (Ahmed et al., 2017).
LIAR	not an abbreviation	A fact-checking dataset named after its paper, "Liar, Liar Pants on Fire" (Wang, 2017). 12,836 short political statements rated by PolitiFact.
McIntire	not an abbreviation	The "fake or real news" dataset compiled by George McIntire: 6,335 full articles (6,059 after cleaning); real ones from mainstream outlets, not Reuters.
E1 … E5	Experiment 1 … 5	Our own labels for the experimental conditions (E2a, E3b etc. are variants).
RQ	Research Question	RQ1–RQ4 in the report.
O1–O7	Objectives 1–7	The objectives listed in section 4.1 of the report.
ML	Machine Learning	Programs that learn a rule from examples instead of being given the rule.
NLP	Natural Language Processing	Computing with human language text.
TF	Term Frequency	How often a word occurs in one document.
IDF	Inverse Document Frequency	How rare a word is across all documents; rare words get a higher weight.
TF-IDF	Term Frequency – Inverse Document Frequency	TF × IDF: turns each article into a list of numbers, one per vocabulary word. See section 4.1.
LR	Logistic Regression	The main classifier: a weighted sum of features turned into a probability. See 4.2.
SVM	Support Vector Machine	Another linear classifier, used as a check. See 4.4.
LinearSVC	Linear Support Vector Classifier	scikit-learn's name for its linear SVM class.
RF	Random Forest	Many decision trees voting; used for the numeric (non-text) probes. See 4.3.
POS	Part of Speech	The grammatical category of a word: noun, verb, determiner, etc.
NER	Named Entity Recognition	Finding names of people, places, organisations, dates etc. in text.
VADER	Valence Aware Dictionary and sEntiment Reasoner	A rule-based sentiment scorer for English (in NLTK). Gives negative, neutral, positive and compound scores.
NLTK	Natural Language Toolkit	Python NLP library; we use only its VADER sentiment scorer.
spaCy	not an abbreviation	Python NLP library; we use it for POS tagging and NER.
en_core_web_sm	English · core pipeline · trained on web text · small	The spaCy model we load.

Short form	In full	What it means here
sklearn	scikit-learn	The Python machine-learning library used for TF-IDF, the classifiers, splitting and metrics.
NumPy	Numerical Python	Library for arrays of numbers and maths on them.
pandas	from "panel data"	Library for tables (DataFrames) and columns (Series).
SD	Standard Deviation	How spread out a set of numbers is. "98.5% ± 0.2%" means the five runs varied by about 0.2 points.
F1	F1 score (F-measure with β = 1)	The harmonic mean of precision and recall; the 1 means both are weighted equally. See 4.5.
TP / FP / TN / FN	True Positive / False Positive / True Negative / False Negative	The four cells of the confusion matrix. "Positive" = predicted fake here.
CSV / TSV	Comma- / Tab-Separated Values	Plain-text table formats. ISOT and McIntire are CSV, LIAR is TSV.
JSON	JavaScript Object Notation	Plain-text format for structured data; every experiment saves its metrics as JSON.
PNG / PDF	Portable Network Graphics / Portable Document Format	Image format of the figures / this document.
URL	Uniform Resource Locator	A web address. Removed from articles in E3b.
HTTP(S)	HyperText Transfer Protocol (Secure)	How web pages are fetched; also appears as "https" in article text.
SHA-256	Secure Hash Algorithm, 256-bit	Produces a 64-character fingerprint of a file; any change to the file changes it. Used to verify the datasets.
MD5	Message Digest 5	An older, faster fingerprint function; used only to label the caches, not for security.
regex	regular expression	A pattern language for matching text, e.g. `@\w+` matches Twitter handles.
NaN	Not a Number	pandas' marker for a missing value.
venv / .venv	virtual environment	A private copy of Python with the project's exact library versions.
pip	"pip installs packages"	Python's package installer.
CPU / RAM	Central Processing Unit / Random-Access Memory	Processor and working memory.
UTF-8	Unicode Transformation Format, 8-bit	The standard way to store any character (curly quotes, "→") as bytes.
cp1252	Windows code page 1252	Windows' old default character set; it cannot print some characters, hence `PYTHONUTF8=1`.
GOP	Grand Old Party	Nickname of the US Republican Party; a strong "fake" token.
EDT	Eastern Daylight Time	US time zone; appears in Reuters timestamps.
NYT / NPR	The New York Times / National Public Radio	Mainstream outlets among McIntire's real articles.

Short form	In full	What it means here
BERT	Bidirectional Encoder Representations from Transformers	A large pretrained neural language model; often suggested as "why not use this?". See section 10.
NLI	Natural Language Inference	The task where Gururangan et al. first showed partial-input artifacts (SNLI = Stanford NLI, MultiNLI = Multi-Genre NLI).
DOI	Digital Object Identifier	Permanent ID for a publication, e.g. 10.18653/v1/N18-2017.
arXiv	pronounced "archive"	Free preprint server; papers there are not necessarily peer reviewed (Verma 2026 is a preprint).

3.2 Part-of-speech tags (spaCy, Universal Dependencies tag set)

E2b, E3c and E5c keep only the tags marked "kept". Everything else is thrown away before the model sees the text.

Tag	In full	Examples	Function-word probe
DET	Determiner	the, a, this, these, every	kept
PRON	Pronoun	he, you, it, we, anyone	kept
ADP	Adposition (preposition)	of, in, on, via, amid	kept
AUX	Auxiliary verb	is, was, has, would, could	kept
CCONJ	Coordinating conjunction	and, but, or	kept
SCONJ	Subordinating conjunction	whether, because, if, that	kept
PART	Particle	not, to (in "to go"), 's	kept
PUNCT	Punctuation	. , ! ? : - ( )	kept
NOUN / PROPN	Noun / Proper noun	minister, budget / Trump, Reuters	dropped
VERB / ADJ / ADV	Verb / Adjective / Adverb	said, voted / huge / sharply	dropped
NUM	Numeral	2018, 14	dropped

3.3 Named-entity types (spaCy), used in E2e

Label	Meaning
PERSON	People, including fictional
ORG	Organisations: companies, agencies, parties
GPE	Geo-Political Entity: countries, cities, states
NORP	Nationalities Or Religious or Political groups (e.g. Republicans, Muslim)
LOC	Non-GPE locations: mountains, seas
FAC	Facilities: buildings, airports, bridges
DATE / TIME	Dates and periods / times of day
CARDINAL	Numbers that are not another type
ORDINAL	first, second …
MONEY / PERCENT / QUANTITY	Amounts with units

Label	Meaning
EVENT / LAW / WORK_OF_ART / PRODUCT / LANGUAGE	Named events, laws, titles of works, products, languages

3.4 Terms

Term	Meaning
Dataset artifact	A pattern in a dataset that predicts the label but has nothing to do with the task. "(Reuters)" appears only in real articles: an artifact.
Construction artifact	An artifact created by how the dataset was built, e.g. collecting each class from a different source.
Shortcut learning	A model solving a task through an easy, irrelevant cue instead of the intended skill (Geirhos et al., 2020).
Partial-input baseline / probe	A model deliberately given only part of the input. If it still does well, that part leaks the label. Our core method.
Leakage	Information reaching the model that should not, making scores look better than the model really is. Also: test-set information leaking into training.
Provenance / source marker	Text that reveals who published the article: the agency name, datelines, image credits, URLs.
Dateline	The place-and-agency prefix of a wire story, e.g. "WASHINGTON (Reuters) - ".
Wire service / newswire	An agency (like Reuters) that writes news for other outlets to republish; has a strict house style.
Page furniture	Text on a web page that is not article prose: image credits, "[Video]" tags, embedded links, handles.
Typography	The characters used for punctuation: curly ' " versus straight ' " quotes, en/em dashes, "…".
Function words	Grammatical words carrying little meaning (the, of, is, and) plus punctuation. Opposite: content words (nouns, verbs, adjectives, names).
Register / genre	The style of writing typical of a setting: wire reporting ("said", "told reporters") vs. opinion blogging ("this", "just", "you").
Ablation	Removing a component and re-measuring, to see what it contributed. E3/E3b are ablations.
Confound	A third factor that could explain a result. Topic is a possible confound here (real has world news; fake does not); E5 removes it.
Topic-matched	Restricted so both classes cover the same topic (E5: US politics only).
Cross-corpus evaluation / transfer	Train on one dataset, test on another. The honest test of whether a model learned the task or the dataset.
In-domain control	The same model tested on its own dataset, to separate "crossing datasets is hard" from "this kind of input is weak".
Chance / majority baseline	Accuracy of always guessing the most common class in the test set. Measured, not assumed to be 50%.
Train / test split	Training data is used to fit the model; the held-out test data is used only to measure it.
Stratified split	A split that keeps the real/fake proportions identical in train and test.

Term	Meaning
Random seed / random_state	A number that fixes the random choices so the same split comes out every time.
Feature	One input number the model sees, e.g. the TF-IDF weight of "said", or the word count.
Vocabulary	The set of distinct words TF-IDF found in the training data; one feature per word.
Token / tokenisation	A unit of text (word or punctuation mark) / splitting text into those units.
Coefficient / weight	The number logistic regression multiplies a feature by. Positive pushes towards fake, negative towards real.
Regularisation	A penalty on large weights so the model does not memorise the training data.
Standardisation	Rescaling a numeric feature to mean 0 and SD 1 so features with big values don't dominate.
Feature importance	In a random forest, how much each feature helped the trees separate the classes, summing to 1.
Confusion matrix	2 × 2 table of actual vs. predicted class. See 4.5.
Accuracy / precision / recall	Share of all predictions that are right / of articles called fake, the share really fake / of fake articles, the share caught. See 4.5.
Generalisation	Doing well on data unlike the training data.
Pipeline (scikit-learn)	A chain of steps (e.g. TF-IDF then classifier) that is fitted and applied as one object, so the steps can't be applied in the wrong order.
Fit / predict	Learn from data / apply what was learned to new data.
Clone	A fresh, untrained copy of a model with the same settings.
Cache	A saved result so slow work (spaCy, VADER) is done once.
Hash / checksum	A short fingerprint computed from data; identical data gives an identical fingerprint.
Process / thread / parallel	Independent running program / a lighter worker inside one program / doing several things at the same time on different CPU cores.
DataFrame / Series	pandas' table / single column.

3.5 Abbreviations in the references

Short form	In full	Where it appears
ACL	Association for Computational Linguistics	The main NLP society; also its annual conference.
NAACL-HLT	North American Chapter of the ACL: Human Language Technologies	Conference (Gururangan et al. 2018).
EACL	European Chapter of the ACL	Conference (Zhou et al. 2021).
EMNLP	Empirical Methods in Natural Language Processing	Conference (Schuster et al. 2019; Gardner et al. 2020 in "Findings of EMNLP").
IJCNLP	International Joint Conference on Natural Language Processing	Held jointly with EMNLP 2019.
***SEM**	Joint Conference on Lexical and Computational Semantics	Conference (Poliak et al. 2018).
ICLR	International Conference on Learning Representations	Conference (Kaushik et al. 2020).

Short form	In full	Where it appears
SIGKDD	Special Interest Group on Knowledge Discovery and Data Mining (of the ACM)	Publishes the Explorations newsletter (Shu et al. 2017).
ACM	Association for Computing Machinery	Computing society.
ISDDC	International Conference on Intelligent, Secure and Dependable Systems in Distributed and Cloud Environments	Where the ISOT paper appeared (Ahmed et al. 2017).
LNCS	Lecture Notes in Computer Science	Springer's book series that published the ISDDC proceedings.

4. Core ideas you need before reading the code

4.1 TF-IDF: turning text into numbers

A classifier only understands numbers. `TfidfVectorizer` first learns a **vocabulary** from the training articles (every distinct word; with default settings, words of two or more letters, lowercased). Each article then becomes one long row of numbers, one per vocabulary word, almost all zero.

- **TF (term frequency):** how many times the word occurs in this article.
- **IDF (inverse document frequency):** scikit-learn uses `ln((1 + n) / (1 + df)) + 1`, where `n` is the number of training articles and `df` the number containing the word. A word in every article ("the") gets the minimum weight; a rare word gets a high one.
- The value for a word is TF × IDF, and each article's row is then scaled to length 1, so a long article does not get bigger numbers just for being long.

Two choices matter for this project. We use **single words** (no word pairs), and we **do not remove stop words** like "the", "on", "said". Many tutorials remove them; we keep them because the style signal lives exactly there.

4.2 Logistic regression (the detector)

Logistic regression gives every feature a **weight** (coefficient). For an article it computes `score = bias + Σ weight × feature value` and turns the score into a probability with the logistic (sigmoid) function `1 / (1 + e^(-score))`. Probability above 0.5 → predicted fake.

Training chooses the weights that make the training labels most probable, with a penalty on large weights (L2 regularisation, scikit-learn's default strength `C = 1`) so it doesn't memorise.

Why we chose it:

- **It is readable.** Each word has exactly one weight. A large positive weight means "this word pushes towards fake"; large negative means "towards real". That is how we found "reuters", "said", "via" and "featured" (report Figure 3). A neural network would not show this so directly.
- **It is the standard baseline** for TF-IDF text classification and the kind of model behind the published 98–99% ISOT results.
- **If a simple linear model already gets 98%, the task is suspiciously easy.** That is the point we are making.

4.3 Random forest (for the numeric probes)

E2c, E2d, E2e and E5d give the model a small table of numbers (8 to 18 columns), not text. For those we use a **random forest**: 100 decision trees (scikit-learn's default), each trained on a random resample of the training data and considering a random subset of features at each split; the trees vote.

Why not logistic regression here? With so few features, the useful rules are likely to be **thresholds and combinations** ("more than two question marks *and* short words"), which trees capture and a single weighted sum does not. The forest also reports **feature importances**: how much each feature helped, summing to 1.

4.4 Linear SVM (the classifier check)

A linear Support Vector Machine also draws a straight boundary, but chooses the one with the widest gap (margin) between the classes. We rerun four conditions with it to answer "is this just because you used logistic regression?". It scores slightly *higher* every time.

4.5 Metrics, using the E1 confusion matrix

Fake is the **positive** class (label 1), real is 0. The E1 test set has 7,729 articles:

	Predicted real	Predicted fake
Actually real (4,238)	4,192 (TN: correct)	46 (FP: real wrongly called fake)
Actually fake (3,491)	75 (FN: fake missed)	3,416 (TP: correct)

- **Accuracy** = correct / all = (4,192 + 3,416) / 7,729 = **98.4%**.
- **Precision** (fake) = TP / (TP + FP) = 3,416 / 3,462 = 98.7%: when it says fake, how often it's right.
- **Recall** (fake) = TP / (TP + FN) = 3,416 / 3,491 = 97.9%: of the fake articles, how many it catches.
- **F1** = 2 × P × R / (P + R) = **0.983**: one number balancing the two.
- **Chance (majority baseline)** = share of the biggest class in the test set = 4,238 / 7,729 = **54.8%**. A model that always says "real" gets this. We **measure** it because it is not 50%: after cleaning, ISOT is 54.8% real, the E5 subset is 60.9% fake, LIAR is 55.8% "real". "Far above chance" is the evidence of leakage, so chance must be right.

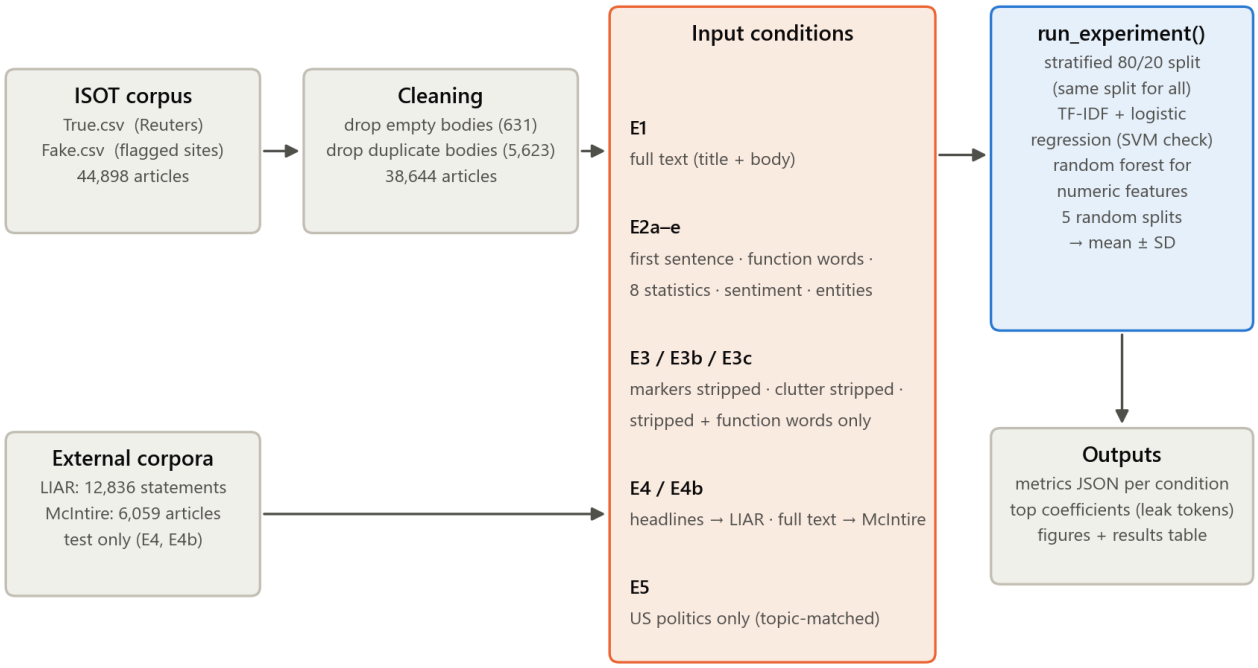
4.6 Train/test split, stratification, seeds and repeats

- **80/20 split:** 80% of articles train the model, the other 20% are only used to measure it. A model is never scored on articles it trained on. 20% of 38,644 is 7,729 test articles, which is enough that one percentage point is about 77 articles.
- **Stratified:** both parts keep the real/fake ratio of the whole data (54.8 / 45.2). Without it a random split could put proportionally more fake articles in the test set by chance, shifting both accuracy and chance for reasons unrelated to the model.
- **Fixed seed (`random_state = 42`):** the same articles land in the test set every run, on every computer, **in every experiment**. So when E1 scores 98.4% and E3b 97.3%, the difference comes from the input, not from a luckier split. (42 is just a convention; any fixed number works.)
- **Five repeats (seeds 42–46):** each experiment is also run on four further random splits. The report gives mean ± SD. SDs are 0.1–0.5 points, so differences of several points between conditions are real, not split luck. Seed 42's run supplies the headline numbers, confusion matrices and coefficients.

4.7 Why a pipeline prevents leakage

TF-IDF *learns* things (the vocabulary, the IDF weights). If it learned them from all the data before splitting, information about the test articles would leak into training. A scikit-learn `Pipeline` fits TF-IDF and the classifier **on the training part only**, then merely *applies* them to the test part. The same goes for `StandardScaler` on numeric features.

5. How the files fit together



Data flows left to right: ISOT is cleaned; each experiment turns the cleaned articles into a restricted **view** (functions in `transforms.py`); every view goes through the same `run_experiment()` function (in `common.py`), which splits, trains, scores and writes a JSON file; `07_figures.py` reads only those JSON files. LIAR and McIntire are used only as test sets (plus McIntire's own in-domain control).

File	Role	Run as
`run.bat`	Windows shortcut: runs `run_all.py` with the project's Python	`.\run`
`run_all.py`	Runs steps 0–7, steps 1–6 three at a time	(called by run.bat)
`scripts/common.py`	Paths, constants, `load_isot()`, `run_experiment()`, printing and coefficient saving	imported, never run
`scripts/transforms.py`	Every restricted view: first sentence, function words, metadata, stripping, entities, sentiment, caching	imported, never run
`scripts/00_get_data.py`	Downloads LIAR/McIntire if missing, checks all data files' SHA-256	step 0
`scripts/01_baseline.py`	E1	step 1
`scripts/02_partial_input.py`	E2a, E2b, E2c, body-only	step 2
`scripts/03_stripped.py`	E3, E3b	step 3
`scripts/04_cross_corpus.py`	E4 (LIAR), E4b (McIntire)	step 4
`scripts/05_more_probes.py`	E2d, E2e, E3c, SVM check	step 5
`scripts/06_confounds.py`	E5a–E5d (topic-matched)	step 6
`scripts/07_figures.py`	Seven figures + `results/results_table.md`	step 7

File	Role	Run as
`data/`	True.csv, Fake.csv, liar/, mcintire/, cache/ (saved spaCy and VADER output)	
`results/`	One JSON per experiment, `*_tokens.json` coefficient lists, results table, `before_quote_fix/`	
`report/`	The Word report and the scripts that build it and this guide (not part of the method, not covered line by line)	

6. Worked example: one real and one fake article through every view

These are real rows of the cleaned dataset, put through the actual functions from `transforms.py` when this PDF was built (long texts are cut with "…"). Use them when explaining what each experiment "sees". Notice in the fake article: "tonight s", "Sanders 42.4" (apostrophes lost in scraping), the "Featured image via … Getty Images" credit, "via Raw Story", and the exclamation marks. Notice in the real article: the dateline "WASHINGTON (Reuters) -", the day of the week, the curly quotes “ ”, and attribution ("called himself", "speaking on").

The "(IMAGE/VIDEO)" in the fake title survives stripping: the media-tag rule matches "(Video)" or "[Image]" but not combined tags. Likewise, in the real article the "leading sentence" rule stops at the first full stop, which is the one in "U.S.", so "S. Congress, …" remains. The rules are hand-written and not perfect (they are applied identically to both classes), which the report states as a limitation; E3c and E5c show the result does not depend on them, because function words carry the signal anyway.

A real (Reuters) article — row 0

Title	As U.S. budget fight looms, Republicans flip their fiscal script
Body (as stored)	WASHINGTON (Reuters) - The head of a conservative Republican faction in the U.S. Congress, who voted this month for a huge expansion of the national debt to pay for tax cuts, called himself a “fiscal conservative” on Sunday and urged budget restraint in 2018. In keeping with a sharp pivot under way among Republicans, U.S. Representative Mark Meadows, speaking on CBS’ “Face the Nation,” drew a hard line on federal spending, which lawmakers are bracing to do battle over in January. When they return from the holidays …
E2a — first sentence	WASHINGTON (Reuters) - The head of a conservative Republican faction in the U
E3 — markers stripped (body)	S. Congress, who voted this month for a huge expansion of the national debt to pay for tax cuts, called himself a “fiscal conservative” on Sunday and urged budget restraint in 2018. In keeping with a sharp pivot under way among Republicans, U.S. Representative Mark Meadows, speaking on CBS’ “Face the Nation,” drew a hard line on federal spending, which lawmakers are bracing to do battle over in January. When they return from the holidays on Wednesday, lawmakers will begin trying to pass a federal budget in a fight …
E3b — fully stripped (title + body)	As U.S. budget fight looms, Republicans flip their fiscal script S. Congress, who voted this month for a huge expansion of the national debt to pay for tax cuts, called himself a fiscal conservative on Sunday and urged budget restraint in 2018. In keeping with a sharp pivot under way among Republicans, U.S. Representative Mark Meadows, speaking on CBS Face the Nation, drew a hard line on federal spending, which lawmakers are bracing to do battle over in January. When they return from the holidays on Wednesday, …
E3c — function words of the E3b text	`as , their , who this for a of the to for , himself a on and in . in with a under among , , on the , a on , which are to do over in . when they from the on , will to a in a to be to , as , as the in which will to of . and his a in , while for on that , , , and . the ( ) has been to : we to ... by , , of the but , on the . , are that not , we to the a of to . for a , i where the is . ... you out of s , he . was among who in for their s - , which is to the and about over to the . it to about , on . the would the to , to be off by , to for and the . this is of the ... we in the of the of . i we to be for this for , to , . the , the in than , will the and . , who the , than , in a that or , as t …`

E2c — the eight numbers (unstripped title + body)	`char_count`	4,724
	`word_count`	759
	`sentence_count`	45
	`exclamation_marks`	0
	`question_marks`	0
	`all_caps_word_ratio`	0.014
	`avg_word_length`	5.220
	`digit_ratio`	0.007

A fake article — row 28,000

Title	Clinton Wins Big In New York Democratic Primary (IMAGE/VIDEO)
Body (as stored)	New York loves Hillary Clinton, and they really turned out to show her that in tonight s primary. Her 56.7% to Sanders 42.4 percent means a huge delegate and popular vote win, with Clinton picking up 129 delegates. This also means that she breaks the 7-state winning streak that Bernie Sanders has enjoyed over her. The results were called a mere 40 minutes after voting ended at 9 PM EST.Here is a map of how the delegate counts shook out for tonight s Democratic Primary in New York, via Raw Story:Always classy, Secretary Clinton tried made sure that people understood that there is more that unites us as Democrats, rather than divides us, whether you supported her or Senator Sanders.Congratulations, Madame Secretary! No matter who wins this nomination, vote BLUE in November!Featured image via Justin Sullivan/Getty Images
E2a — first sentence	New York loves Hillary Clinton, and they really turned out to show her that in tonight s primary
E3 — markers stripped (body)	Her 56.7% to Sanders 42.4 percent means a huge delegate and popular vote win, with Clinton picking up 129 delegates. This also means that she breaks the 7-state winning streak that Bernie Sanders has enjoyed over her. The results were called a mere 40 minutes after voting ended at 9 PM EST.Here is a map of how the delegate counts shook out for tonight s Democratic Primary in New York, via Raw Story:Always classy, Secretary Clinton tried made sure that people understood that there is more that unites us as Democrats, rather than divides us, whether you supported her or Senator Sanders.Congratulations, Madame Secretary! No matter who wins this nomination, vote BLUE in November!Featured image via Justin Sullivan/Getty Images
E3b — fully stripped (title + body)	Clinton Wins Big In New York Democratic Primary (IMAGE/VIDEO) Her 56.7% to Sanders 42.4 percent means a huge delegate and popular vote win, with Clinton picking up 129 delegates. This also means that she breaks the 7-state winning streak that Bernie Sanders has enjoyed over her. The results were called a mere 40 minutes after voting ended at 9 PM EST.Here is a map of how the delegate counts shook out for tonight s Democratic Primary in New York, via Raw Story:Always classy, Secretary Clinton tried made sure that people understood that there is more that unites us as Democrats, rather than divides us, whether you supported her or Senator Sanders.Congratulations, Madame Secretary! No matter who wins this nomination, vote BLUE in November!
E3c — function words of the E3b text	`in ( ) her to a and , with up . this that she the - that has over her . the were a after at is a of how the out for s in , via : , that that there is that us as , than us , whether you her or . , ! no who this , in !`

E2c — the eight numbers (unstripped title + body)	`char_count`	893
	`word_count`	146
	`sentence_count`	9
	`exclamation_marks`	2
	`question_marks`	0
	`all_caps_word_ratio`	0.021
	`avg_word_length`	5.110
	`digit_ratio`	0.015

7. Line-by-line walkthrough

Each block shows the code with its real line numbers, then what it does and why. Read `common.py` and `transforms.py` first: the experiment scripts are short because they call these.

7.1 run.bat and run_all.py — the one command

Purpose: let anyone reproduce everything with one command (`.\run`), in about five minutes.

run.bat · Lines 1–2

```
@echo off
"%~dp0.venv\Scripts\python.exe" "%~dp0run_all.py" %*
```

`@echo off` stops Windows printing each command before running it. Line 2 runs the project's own Python (the one inside `.venv`, with the exact library versions from `requirements.txt`) on `run_all.py`. `%~dp0` means "the folder this .bat file is in", so the command works from any folder. `%*` passes on whatever was typed after `run`: `run 3 7` becomes `run_all.py 3 7`.

run_all.py · Lines 1–11

```
"""Run the whole pipeline, or chosen steps.

    run              everything: 0 (fetch and verify data), then 1-6, then 7 (figures)
    run 2            only step 2
    run 3 7          steps 3 and 7, in that order

With no arguments, steps 1-6 run three at a time once the spaCy/VADER caches in
data/cache/ exist (they are independent of each other); their output goes to
results/logs/step_N.log. Without the caches they run one at a time, because each
cache is built by whichever step needs it first. Results are identical either way.
"""
```

The **docstring** (text in triple quotes at the top of a file or function) documents usage. It also records a design decision: steps 1–6 only run in parallel once the caches exist (explained at lines 33–35).

run_all.py · Lines 13–18

```
import os
import subprocess
import sys
import time
from concurrent.futures import ThreadPoolExecutor, as_completed
from pathlib import Path
```

Imports. `os` gives access to environment variables; `subprocess` starts another program (each step runs as its own Python process); `sys` reads the command-line arguments and exits with an error; `time` measures durations; `ThreadPoolExecutor` and `as_completed` run several steps at once and collect them as they finish; `Path` handles file paths on any operating system.

run_all.py · Lines 20–32

```
ROOT = Path(__file__).resolve().parent
SCRIPTS = ROOT / "scripts"
LOGS = ROOT / "results" / "logs"
STEPS = {
    "0": "00_get_data.py",
    "1": "01_baseline.py",
    "2": "02_partial_input.py",
    "3": "03_stripped.py",
    "4": "04_cross_corpus.py",
    "5": "05_more_probes.py",
    "6": "06_confounds.py",
    "7": "07_figures.py",
}
```

`Path(__file__).resolve().parent` is the folder containing this file, so all paths are relative to the project wherever it is copied. `STEPS` maps each step number to its script.

run_all.py · Lines 33–35

```
33  INDEPENDENT = ["5", "4", "6", "3", "2", "1"]  # steps 1-6, longest first
34  CACHES = ["function_words", "stripped_profile", "sentiment", "mcintire_profile"]
35  PARALLEL_STEPS = 3  # each step already fits 5 splits in parallel; 3 x 5 suits a 16-core machine
```

- `INDEPENDENT`: steps 1–6 never read each other's output, so they can run at the same time. They are listed **longest first** so the slow steps start immediately and the total time is shortest.
- `CACHES`: the four saved spaCy/VADER results. If two steps both found a cache missing at the same time, both would spend ~25 minutes building it and could overwrite each other's file. So parallel mode is only used once all four exist.
- `PARALLEL_STEPS = 3`: each experiment already fits its 5 splits in parallel, so 3 steps × 5 fits = 15 workers, which suits a 16-core laptop.

run_all.py · Lines 37–39

```
37  # UTF-8 mode: token lists contain characters (curly quotes, '…', '→') that the
38  # default Windows codepage cannot print.
39  ENV = {**os.environ, "PYTHONUTF8": "1"}
```

Copies the current environment variables and adds `PYTHONUTF8=1`. On Windows, Python otherwise prints with code page 1252, which cannot show characters such as curly quotes or "→" that appear in token lists, and the step would crash.

run_all.py · Lines 42–51

```
42  def run_step(step, log_file=None):
43      """Run one step; return (step, seconds, return code)."""
44      start = time.perf_counter()
45      cmd = [sys.executable, str(SCRIPTS / STEPS[step])]
46      if log_file is None:
47          code = subprocess.run(cmd, env=ENV).returncode
48      else:
49          with open(log_file, "w", encoding="utf-8") as log:
50              code = subprocess.run(cmd, env=ENV, stdout=log, stderr=subprocess.STDOUT).returncode
51      return step, time.perf_counter() - start, code
```

`run_step` runs one script. It starts a timer, builds the command `[python, script path]` (`sys.executable` is the Python running this file, i.e. the venv's), and runs it. With no log file the output goes to the screen; otherwise standard output and errors both go into the log file. It returns the step, the seconds taken and the **return code**: 0 means success, anything else failure.

run_all.py · Lines 54–60

```
54  def run_in_order(steps, timings):
55      for step in steps:
56          print(f"\n{'=' * 60}\n[{step}] {STEPS[step]}\n{'=' * 60}", flush=True)
57          step, seconds, code = run_step(step)
58          if code != 0:
59              sys.exit(f"step {step} ({STEPS[step]}) failed; stopping.")
60          timings.append((step, seconds))
```

Runs steps one after another, printing a header for each. If a step fails, `sys.exit` stops the whole pipeline with a message, because later steps (especially figures) would otherwise silently use missing or old results.

run_all.py · Lines 63–79

```
63  def run_in_parallel(steps, timings):
64      LOGS.mkdir(parents=True, exist_ok=True)
65      print(f"\nrunning steps {', '.join(sorted(steps))} ({PARALLEL_STEPS} at a time); "
66            f"output in {LOGS.relative_to(ROOT)}", flush=True)
67      failed = []
68      with ThreadPoolExecutor(max_workers=PARALLEL_STEPS) as pool:
69          futures = [pool.submit(run_step, s, LOGS / f"step_{s}.log") for s in steps]
70          for future in as_completed(futures):
71              step, seconds, code = future.result()
72              status = "done" if code == 0 else "FAILED"
73              print(f"  [{step}] {STEPS[step]:<22} {status} in {seconds / 60:.1f} min", flush=True)
74              if code == 0:
75                  timings.append((step, seconds))
76              else:
77                  failed.append(step)
78      if failed:
79          sys.exit(f"step(s) {', '.join(failed)} failed; see results/logs/step_N.log")
```

Runs steps three at a time. `pool.submit` starts each step on one of 3 worker threads; `as_completed` hands back each step as it finishes, so its time can be printed straight away. Failures are collected and reported at the end. **Threads are enough here** because each thread only waits for a separate Python process; the real work happens in those processes, on their own CPU cores.

run_all.py · Lines 82–86

```
82  def main():
83      chosen = sys.argv[1:]
84      unknown = [s for s in chosen if s not in STEPS]
85      if unknown:
86          sys.exit(f"unknown step(s): {', '.join(unknown)}. Choose from {', '.join(STEPS)}.")
```

`sys.argv[1:]` is the list of words typed after the script name (`run 3 7` → `["3", "7"]`). Unknown step numbers are rejected with a helpful message.

run_all.py · Lines 88–100

```
88      start = time.perf_counter()
89      timings = []
90      if chosen:
91          run_in_order(chosen, timings)
92      else:
93          run_in_order(["0"], timings)
94          caches_ready = all((ROOT / "data" / "cache" / f"{c}.json").exists() for c in CACHES)
95          if caches_ready:
96              run_in_parallel(INDEPENDENT, timings)
97          else:
98              print("\nspaCy/VADER caches not built yet: running steps one at a time this once")
99              run_in_order(sorted(INDEPENDENT), timings)
100         run_in_order(["7"], timings)
```

The plan. If steps were named, run exactly those, in that order. Otherwise: step 0 (check data); then steps 1–6, in parallel if all caches exist, else one at a time (the first ever run, which builds the caches); then step 7 last, because figures need every result.

run_all.py · Lines 102–106

```
    print("\ntime per step:")
    for step, seconds in sorted(timings):
        print(f"  [{step}] {STEPS[step]:<22} {seconds / 60:5.1f} min")
    print(f"  wall-clock total {(time.perf_counter() - start) / 60:.1f} min")
    print("\ndone. metrics in results/, charts in figures/")
```

Prints how long each step took and the total.

run_all.py · Lines 109–110

```
if __name__ == "__main__":
    main()
```

`if __name__ == "__main__":` is true only when the file is run directly, not when it is imported by another file. Every script ends this way.

7.2 scripts/common.py — the shared experiment harness

Purpose: one place that loads and cleans ISOT, and one function, `run_experiment()`, that every experiment calls. This is the most important file to understand. Because every condition goes through the same function, with the same split and the same classifier, **the only thing that differs between experiments is the input**. That is what makes the comparisons fair.

scripts/common.py · Line 1

```
1   """Shared experiment harness. Every experiment goes through run_experiment()."""
```

Module docstring.

scripts/common.py · Lines 3–15

```
3   import json
4   from pathlib import Path
5
6   import numpy as np
7   import pandas as pd
8   from joblib import Parallel, delayed
9   from sklearn.base import clone
10  from sklearn.feature_extraction.text import TfidfVectorizer
11  from sklearn.linear_model import LogisticRegression
12  from sklearn.metrics import accuracy_score, confusion_matrix, precision_recall_fscore_support
13  from sklearn.model_selection import train_test_split
14  from sklearn.pipeline import Pipeline
15  from sklearn.preprocessing import StandardScaler
```

Imports:

- `json` writes the results files; `Path` builds file paths.
- `numpy` (as `np`) for arrays and maths; `pandas` (as `pd`) for tables.
- `Parallel, delayed` from **joblib** run the five splits at once on separate CPU cores.
- `clone` makes a fresh, untrained copy of a model.
- `TfidfVectorizer` turns text into TF-IDF numbers; `LogisticRegression` is the classifier.
- `accuracy_score`, `confusion_matrix`, `precision_recall_fscore_support` compute the metrics.
- `train_test_split` makes the 80/20 split; `Pipeline` chains steps; `StandardScaler` rescales numbers.

scripts/common.py · Lines 17–19

```
17  ROOT = Path(__file__).resolve().parent.parent
18  DATA_DIR = ROOT / "data"
19  RESULTS_DIR = ROOT / "results"
```

Project paths. `__file__` is this file; `.parent.parent` goes up from `scripts/` to the project folder.

scripts/common.py · Lines 21–24

```
21  RANDOM_STATE = 42
22  TEST_SIZE = 0.2
23  N_SPLITS = 5  # random splits per experiment for the mean +/- SD; seeds 42..46
24  REAL, FAKE = 0, 1
```

Constants used everywhere, so they cannot differ between experiments:

- `RANDOM_STATE = 42`: the seed that fixes the split (section 4.6).
- `TEST_SIZE = 0.2`: 20% test, 80% train. 80/20 is the conventional choice; with 38,644 articles, 7,729 test articles make each percentage point about 77 articles, so estimates are stable.

- `N_SPLITS = 5`: number of random splits per experiment (seeds 42–46).
- `REAL, FAKE = 0, 1`: fake is class 1, the "positive" class, so precision, recall and F1 describe how well **fake** articles are detected.

scripts/common.py · Lines 27–37

```
27  def load_isot():
28      """Load ISOT as one DataFrame: True.csv rows first, then Fake.csv rows.
29
30      Cleaning: drop rows with an empty body (630 of the 631 are fake, so an empty
31      body is itself a label leak) and drop duplicate bodies, keeping the first copy
32      (duplicates split across train/test would let the model be tested on articles
33      it has already seen).
34
35      Every experiment loads through here so row order is identical everywhere,
36      which (with the fixed random_state) gives every experiment the same split.
37      """
```

`load_isot()`'s docstring states the two cleaning rules and why. Every experiment loads data through this function, so the row order is identical everywhere; together with the fixed seed, every experiment gets the **same test articles**.

scripts/common.py · Lines 38–41

```
38      true = pd.read_csv(DATA_DIR / "True.csv")
39      fake = pd.read_csv(DATA_DIR / "Fake.csv")
40      true["label"] = REAL
41      fake["label"] = FAKE
```

Reads both CSV files (21,417 real and 23,481 fake rows) and adds a `label` column: 0 for every row of True.csv, 1 for every row of Fake.csv. The label comes only from which file the row is in. That is the root of the problem: the label is really "which source", and the model can learn exactly that.

scripts/common.py · Lines 42–46

```
42      df = pd.concat([true, fake], ignore_index=True)
43      df[["title", "text"]] = df[["title", "text"]].fillna("")
44      df = df[df["text"].str.strip() != ""]
45      df = df.drop_duplicates(subset="text", keep="first")
46      return df.reset_index(drop=True)
```

- Line 42: stack the two tables, real rows first. `ignore_index=True` renumbers rows 0, 1, 2 …
- Line 43: `fillna("")` replaces missing titles or bodies (NaN) with empty text so string operations don't fail.
- Line 44: **drop articles with an empty body** (631 rows). 630 of them are fake, so keeping them would hand the model a free rule, "empty = fake": a leak we would have created ourselves.
- Line 45: **drop duplicate bodies**, keeping the first copy (5,623 rows, almost all fake). If the same article sat in both train and test, the model would be "tested" on something it had memorised, inflating accuracy.
- Line 46: renumber the rows 0 … 38,643. Result: **38,644 articles, 21,191 real and 17,453 fake**.

scripts/common.py · Lines 49–58

```
49  def _fit_on_split(pipeline, features, labels, seed):
50      """Fit a fresh copy of the pipeline on one stratified 80/20 split.
51
52      Returns (fitted pipeline, training-set size, test labels, test predictions).
53      """
54      X_train, X_test, y_train, y_test = train_test_split(
55          features, labels, test_size=TEST_SIZE, stratify=labels, random_state=seed
56      )
57      fitted = clone(pipeline).fit(X_train, y_train)
58      return fitted, len(y_train), y_test, fitted.predict(X_test)
```

`_fit_on_split` (the leading underscore marks a helper meant for use only inside this file) does one train/test run:

- `train_test_split(..., test_size=TEST_SIZE, stratify=labels, random_state=seed)` returns training features, test features, training labels and test labels. `stratify=labels` keeps the real/fake ratio identical in both parts (section 4.6).
- `clone(pipeline).fit(X_train, y_train)` trains a **fresh copy**, so the five runs cannot affect each other. Fitting the pipeline means: learn the TF-IDF vocabulary from the training text only, transform it, then train the classifier.
- It returns the trained copy, the training-set size, the true test labels and the predicted labels.

scripts/common.py · Lines 61–74

```
61  def run_experiment(features, labels, name, model=None, tfidf_kwargs=None,
62                     test_features=None, test_labels=None):
63      """Split, fit, evaluate, save metrics to results/{name}.json.
64
65      features: a 1-D sequence of strings (list / Series) -> TF-IDF is fitted on it;
66                or a 2-D numeric matrix (ndarray / DataFrame) -> used as-is, standardised.
67      model:    classifier to use; defaults to logistic regression.
68      tfidf_kwargs: passed to TfidfVectorizer (text input only).
69      test_features / test_labels: an external test set (cross-corpus, E4). When given,
70                there is no split: the model trains on all of features/labels.
71
72      Returns the metrics dict plus a "pipeline" key holding the fitted model
73      (the pipeline is not written to JSON).
74      """
```

The signature and docstring of `run_experiment`. Parameters:

- `features`: either **text** (one string per article) or a **numeric table** (one row per article, one column per measurement).
- `labels`: 0/1 per article. `name`: used for the results file name, e.g. `e1_baseline`.
- `model`: the classifier; logistic regression if none is given (the SVM and random forest runs pass their own).
- `tfidf_kwargs`: extra settings for TF-IDF (used by the function-word probes, see transforms.py line 23).
- `test_features` / `test_labels`: for cross-corpus tests (E4, E4b). If given, there is no split: the model trains on all of `features` and is tested on this other dataset.

scripts/common.py · Lines 75–76

```
75      labels = np.asarray(labels)
76      is_matrix = getattr(features, "ndim", 1) == 2
```

- `np.asarray` turns the labels into a NumPy array, whatever form they arrived in.

- `is_matrix`: a numeric table has 2 dimensions (rows × columns), a list of strings has 1. `getattr(features, "ndim", 1)` reads the number of dimensions, and gives 1 for a plain Python list, which has no `ndim` attribute. So the function decides by itself whether it was given text or numbers.

scripts/common.py · Lines 78–82

```
78      clf = model if model is not None else LogisticRegression(max_iter=1000)
79      if is_matrix:
80          pipeline = Pipeline([("scale", StandardScaler()), ("clf", clf)])
81      else:
82          pipeline = Pipeline([("tfidf", TfidfVectorizer(**(tfidf_kwargs or {}))), ("clf", clf)])
```

- Line 78: use the given model, or else `LogisticRegression(max_iter=1000)`. `max_iter` is the number of steps the training algorithm may take; the default 100 can stop before training finishes on tens of thousands of features (and print a warning). 1000 gives it room.
- Line 80, numbers: `StandardScaler` (mean 0, SD 1 per column, so `char_count` in the thousands doesn't swamp `digit_ratio` around 0.01), then the classifier.
- Line 82, text: `TfidfVectorizer` then the classifier. `**(tfidf_kwargs or {})` passes any extra TF-IDF settings, or none.
- Both are **Pipelines**, so TF-IDF and scaling are fitted on the training part only (section 4.7).

scripts/common.py · Lines 84–92

```
84      if test_features is None:
85          # The fixed split (seed 42) gives the headline metrics and confusion matrix; N_SPLITS - 1
86          # further random stratified splits give accuracy as mean +/- SD. The fits are
independent,
87          # so they run in parallel; results are identical to fitting them one after another.
88          seeds = range(RANDOM_STATE, RANDOM_STATE + N_SPLITS)
89          fits = Parallel(n_jobs=N_SPLITS)(
90              delayed(_fit_on_split)(pipeline, features, labels, seed) for seed in seeds
91          )
92          pipeline, n_train, y_test, y_pred = fits[0]
```

The normal case, no external test set:

- `seeds = range(42, 47)`: five seeds, 42 to 46.
- `Parallel(n_jobs=5)(delayed(_fit_on_split)(...) for seed in seeds)` runs `_fit_on_split` once per seed, all five at the same time on separate CPU cores. `delayed` packages each call so joblib can hand it to a worker. Results come back in seed order and are identical to running them one by one, just about five times faster.
- Line 92: the seed-42 run (`fits[0]`) supplies the headline numbers, the confusion matrix and the trained model whose coefficients we read. The other four only feed the mean ± SD.

scripts/common.py · Lines 93–97

```
93      else:
94          # A fixed external test set (cross-corpus): train on everything, nothing to resplit.
95          fits = None
96          pipeline.fit(features, labels)
97          n_train, y_test, y_pred = len(labels), np.asarray(test_labels),
pipeline.predict(test_features)
```

Cross-corpus case: the test set is a different dataset, fixed in advance, so there is nothing to re-split. Train once on **all** of the training data and predict the other dataset. No SD is reported for these rows ("n/a, fixed test set" in the table).

scripts/common.py · Lines 99–101

```
99      precision, recall, f1, _ = precision_recall_fscore_support(
100         y_test, y_pred, average="binary", pos_label=FAKE, zero_division=0
101     )
```

Precision, recall and F1 for the fake class: `average="binary"` with `pos_label=FAKE` means "treat fake as the positive class". `zero_division=0` returns 0 instead of a warning if a model never predicts fake. The fourth returned value (support, i.e. counts) is not needed, hence `_`.

scripts/common.py · Lines 102–115

```
102     metrics = {
103         "name": name,
104         "input": "feature_matrix" if is_matrix else "text",
105         "model": type(clf).__name__,
106         "n_train": int(n_train),
107         "n_test": int(len(y_test)),
108         "majority_baseline": float(np.bincount(y_test).max() / len(y_test)),
109         "accuracy": float(accuracy_score(y_test, y_pred)),
110         "precision": float(precision),
111         "recall": float(recall),
112         "f1": float(f1),
113         # rows = true [real, fake], columns = predicted [real, fake]
114         "confusion_matrix": confusion_matrix(y_test, y_pred, labels=[REAL, FAKE]).tolist(),
115     }
```

The metrics saved for every experiment:

- `input`: whether it was text or a feature matrix; `model`: the classifier's class name.
- `n_train`, `n_test`: sizes. `int(...)` and `float(...)` convert NumPy numbers into plain Python numbers, which JSON can store.
- `majority_baseline`: `np.bincount(y_test)` counts how many test labels are 0 and how many are 1; the larger count divided by the total is the accuracy of always guessing that class = **chance**.
- `accuracy`, `precision`, `recall`, `f1`: as in section 4.5.
- `confusion_matrix`: rows are the true class [real, fake], columns the predicted class. `labels=[REAL, FAKE]` fixes that order. `.tolist()` makes it JSON-friendly.

scripts/common.py · Lines 117–121

```
117     if fits is not None:
118         accuracies = [float(accuracy_score(y_te, y_pr)) for _, _, y_te, y_pr in fits]
119         metrics["accuracy_splits"] = accuracies
120         metrics["accuracy_mean"] = float(np.mean(accuracies))
121         metrics["accuracy_sd"] = float(np.std(accuracies, ddof=1))
```

Only when there were five splits: the five accuracies, their mean and their standard deviation. `ddof=1` gives the **sample** standard deviation (divide by n − 1), the correct one for a small sample.

scripts/common.py · Lines 123–127

```
123     RESULTS_DIR.mkdir(exist_ok=True)
124     with open(RESULTS_DIR / f"{name}.json", "w") as f:
125         json.dump(metrics, f, indent=2)
126
127     return {**metrics, "pipeline": pipeline}
```

Writes `results/<name>.json` (creating `results/` if needed). Every number in the report and the figures comes from these files. The function returns the metrics plus the trained pipeline; the pipeline isn't saved to JSON (a model object isn't text), but the caller can read its coefficients or feature importances.

scripts/common.py · Lines 130–140

```
130 def print_metrics(result):
131     print(f"{result['name']}  (n_train={result['n_train']}, n_test={result['n_test']})")
132     print(f"  majority baseline {result['majority_baseline']:.4f}")
133     for metric in ("accuracy", "precision", "recall", "f1"):
134         print(f"  {metric:<17} {result[metric]:.4f}")
135     if "accuracy_mean" in result:
136         print(f"  accuracy over {len(result['accuracy_splits'])} splits  "
137               f"{result['accuracy_mean']:.4f} +/- {result['accuracy_sd']:.4f}")
138     print("  confusion matrix (rows true real/fake, cols predicted real/fake):")
139     for row in result["confusion_matrix"]:
140         print("   ", row)
```

`print_metrics` prints a result readably on screen: sizes, chance, the four metrics to 4 decimals, the five-split mean ± SD if present, and the confusion matrix. `f"{metric:<17}"` pads names to 17 characters so the numbers line up.

scripts/common.py · Lines 143–148

```
143 def save_top_tokens(result, k=30):
144     """Print and save the k strongest logistic regression coefficients per class.
145 
146     Positive coefficients push towards class 1 (fake), negative towards class 0 (real).
147     Written to results/{name}_tokens.json for the figures script.
148     """
```

`save_top_tokens`: the strongest words per class. Only works for linear models on text (logistic regression), since only they have one weight per word.

scripts/common.py · Lines 149–156

```
149     pipeline = result["pipeline"]
150     vocab = pipeline.named_steps["tfidf"].get_feature_names_out()
151     coefs = pipeline.named_steps["clf"].coef_[0]
152     order = np.argsort(coefs)
153     tokens = {
154         "fake": [[vocab[i], float(coefs[i])] for i in order[::-1][:k]],
155         "real": [[vocab[i], float(coefs[i])] for i in order[:k]],
156     }
```

- `vocab`: the vocabulary TF-IDF learned, in column order; `vocab[i]` is the word for column `i`.
- `coefs`: the classifier's weights. `coef_` has one row for a two-class problem, hence `[0]`.
- `np.argsort(coefs)` gives the column numbers sorted from most negative to most positive weight.
- The last `k` (reversed, `order[::-1][:k]`) are the strongest **fake** words; the first `k` are the strongest **real** words.

scripts/common.py · Lines 157–164

```
157     for cls in ("fake", "real"):
158         print(f"\nTop {k} tokens -> {cls.upper()}")
159         for token, coef in tokens[cls]:
160             print(f"  {coef:+8.3f}  {token}")
161 
162     with open(RESULTS_DIR / f"{result['name']}_tokens.json", "w") as f:
163         json.dump(tokens, f, indent=2)
164     return tokens
```

Prints the lists with the sign shown (`+8.3f`) and saves `results/<name>_tokens.json`, which `07_figures.py` uses for the token chart (report Figure 3).

7.3 scripts/transforms.py — the restricted views of an article

Purpose: every function here takes articles and returns only what one condition is allowed to see. Keeping them in one file means the same definition is used everywhere (e.g. "fully stripped" means the same in E3b, E4b, E5 and the SVM check). The slow ones save their output to `data/cache/`.

scripts/transforms.py · Lines 1–6

```
"""Text transformations shared by the experiment scripts.

Each turns articles into the restricted view a condition is allowed to see.
The slow ones (spaCy, VADER) cache their output to data/cache/, keyed by a hash
of the input texts, so they run once and are rebuilt only if the data changes.
"""
```

Module docstring: what the file is for and how caching works.

scripts/transforms.py · Lines 8–14

```
import hashlib
import json
import re

import pandas as pd

from common import DATA_DIR
```

`hashlib` computes fingerprints (for cache keys); `json` reads and writes caches; `re` is Python's regular-expression module (pattern matching on text); `pandas` builds feature tables; `DATA_DIR` comes from `common.py`.

scripts/transforms.py · Lines 16–18

```
CACHE_DIR = DATA_DIR / "cache"
SPACY_PROCESSES = 4  # benchmarked: no further speed-up beyond 4 on this machine
FUNCTION_POS = {"DET", "PRON", "ADP", "AUX", "CCONJ", "SCONJ", "PART", "PUNCT"}
```

- `CACHE_DIR = data/cache/`.
- `SPACY_PROCESSES = 4`: spaCy tags texts using 4 processes at once. Tested: more than 4 was not faster on this machine.
- `FUNCTION_POS`: the part-of-speech tags we count as function words: determiners, pronouns, adpositions, auxiliaries, both kinds of conjunction, particles and punctuation (table in section 3.2). Everything else (nouns, verbs, adjectives, adverbs, names, numbers) is thrown away.

scripts/transforms.py · Lines 20–23

```
# Function-word text is lowercased and space-separated. The default TF-IDF token
# pattern would drop one-character tokens ('a', ',', '-'), which are exactly what
# these probes keep, so split on whitespace instead.
WHITESPACE_TFIDF = {"tokenizer": str.split, "token_pattern": None, "lowercase": False}
```

TF-IDF's default tokeniser keeps only words of **two or more** letters or digits, so it would throw away "a", ",", "!", "?", "-" and ":", which are exactly the tokens the function-word probes need. These settings tell it to split on spaces instead (`tokenizer=str.split`, `token_pattern=None`) and not to lowercase (the text is lowercased already).

scripts/transforms.py · Lines 26–44

```
26  # ---------- caching ----------
27
28  def _cached(name, texts, compute):
29      """Return compute(texts), cached at data/cache/{name}.json and keyed by the texts."""
30      path = CACHE_DIR / f"{name}.json"
31      key = hashlib.md5("\x00".join(texts).encode("utf-8")).hexdigest()
32      if path.exists():
33          with open(path, encoding="utf-8") as f:
34              cache = json.load(f)
35          if cache["key"] == key:
36              print(f"{name}: loaded from cache")
37              return cache["docs"]
38          print(f"{name}: cache is stale (data changed), rebuilding")
39
40      docs = compute(texts)
41      CACHE_DIR.mkdir(exist_ok=True)
42      with open(path, "w", encoding="utf-8") as f:
43          json.dump({"key": key, "docs": docs}, f)
44      return docs
```

The cache. `_cached(name, texts, compute)` returns `compute(texts)`, but saves it the first time:

- Line 31: the **key** is an MD5 fingerprint of all the input texts joined together (separated by the null character `\x00`, which never occurs in articles). Any change to any text changes the key.
- Lines 32–38: if a cache file exists and its key matches, load it (seconds instead of ~25 minutes). If the key differs, the data has changed, so the cache is **stale** and is rebuilt. A cache can therefore never silently feed results computed from different data.
- Lines 40–44: otherwise compute, then save `{"key": ..., "docs": ...}` as JSON.

MD5 is used as a fingerprint, not for security, so its known weaknesses don't matter here.

scripts/transforms.py · Lines 47–55

```
47  def _spacy_pipe(texts, disable, describe):
48      import spacy  # imported here so cached runs don't pay spaCy's load time
49
50      print(f"{describe}: running spaCy over {len(texts)} documents (one-off, ~25 min)")
51      nlp = spacy.load("en_core_web_sm", disable=disable)
52      for i, doc in enumerate(nlp.pipe(texts, batch_size=50, n_process=SPACY_PROCESSES)):
53          if (i + 1) % 2000 == 0:
54              print(f"  {i + 1}/{len(texts)}", flush=True)
55          yield doc
```

`_spacy_pipe` runs spaCy over many texts:

- `import spacy` is inside the function so that cached runs never pay spaCy's loading time.
- `spacy.load("en_core_web_sm", disable=...)` loads the small English model with the parts we don't need switched off (faster).
- `nlp.pipe(texts, batch_size=50, n_process=4)` processes the texts in batches of 50 across 4 processes. Progress is printed every 2,000 documents.
- `yield` makes this a **generator**: it hands back one processed document at a time instead of holding all 38,644 in memory.

scripts/transforms.py · Lines 58–59

```
58  def _function_word_string(doc):
59      return " ".join(t.lower_ for t in doc if t.pos_ in FUNCTION_POS)
```

For one spaCy document: keep each token whose POS tag is in `FUNCTION_POS`, lowercased (`t.lower_`), joined by spaces. E.g. "The minister said on Sunday that he would resign." → "the on that he would ."

scripts/transforms.py · Lines 62–66

```
62  # ---------- E2a ----------
63
64  def first_sentence(text):
65      """Everything before the first '.' or newline."""
66      return re.split(r"[.\n]", text, maxsplit=1)[0]
```

E2a. `re.split(r"[.\n]", text, maxsplit=1)[0]` splits the text at the first full stop or line break and keeps the part before it. For Reuters articles this is the dateline plus the opening of the first sentence (section 6). It is deliberately simple; "U.S." would cut early, which doesn't matter for a probe.

scripts/transforms.py · Lines 69–76

```
69  # ---------- E2b ----------
70
71  def function_words(texts):
72      """Reduce each text to its function words, e.g. 'the of a and to , the -'."""
73      def compute(texts):
74          docs = _spacy_pipe(texts, ["parser", "ner", "lemmatizer"], "function words")
75          return [_function_word_string(doc) for doc in docs]
76      return _cached("function_words", texts, compute)
```

E2b. Runs spaCy with the parser, entity recogniser and lemmatiser disabled (only the POS tagger is needed), converts every article to its function-word string and caches the result as `function_words`.

scripts/transforms.py · Lines 79–98

```
79  # ---------- E2c ----------
80
81  def metadata_features(texts):
82      """Eight surface statistics per article. No words survive into the model."""
83      rows = []
84      for t in texts:
85          words = t.split()
86          n_words = max(len(words), 1)
87          n_chars = max(len(t), 1)
88          rows.append({
89              "char_count": len(t),
90              "word_count": len(words),
91              "sentence_count": len(re.findall(r"[.!?]+", t)),
92              "exclamation_marks": t.count("!"),
93              "question_marks": t.count("?"),
94              "all_caps_word_ratio": sum(w.isupper() and len(w) > 1 for w in words) / n_words,
95              "avg_word_length": sum(len(w) for w in words) / n_words,
96              "digit_ratio": sum(c.isdigit() for c in t) / n_chars,
97          })
98      return pd.DataFrame(rows)
```

E2c. Eight surface statistics per article, no words:

- `char_count`, `word_count` (`t.split()` splits on whitespace).
- `sentence_count`: number of runs of `.`, `!` or `?` (`[.!?]+` counts "?!" as one).
- `exclamation_marks`, `question_marks`.
- `all_caps_word_ratio`: share of words written entirely in capitals with more than one letter ("BREAKING", "HILARIOUS"; "I" and "A" don't count).
- `avg_word_length`, `digit_ratio` (share of characters that are digits).

- `max(..., 1)` prevents division by zero for an empty text. Returns a pandas DataFrame, which has 2 dimensions, so `run_experiment` treats it as numbers (scaler + classifier).

scripts/transforms.py · Lines 101–107

```
# ---------- E3 / E3b ----------

# Broader than '^[A-Z ]+\(Reuters\) -' so it also catches 'WASHINGTON/BEIJING (Reuters) -'
# and 'SAN JUAN, Puerto Rico (Reuters) -'. Only matches at the start of the body.
DATELINE = re.compile(r"^[^\n]*?\(Reuters\)\s*-\s*")
LEADING_SENTENCE = re.compile(r"^[^.\n]*[.\n]\s*")
REUTERS = re.compile(r"reuters", re.IGNORECASE)
```

Patterns for **E3**, compiled once with `re.compile` for speed:

- `DATELINE` = `^[^\n]*?\(Reuters\)\s*-\s*` piece by piece: `^` the start of the text; `[^\n]*?` any characters except a line break, as few as possible; `\(Reuters\)` the literal "(Reuters)" (brackets escaped with `\`); `\s*-\s*` a dash with optional spaces around it. Catches "WASHINGTON (Reuters) - ", "WASHINGTON/BEIJING (Reuters) - " and "SAN JUAN, Puerto Rico (Reuters) - ".
- `LEADING_SENTENCE` = `^[^.\n]*[.\n]\s*`: from the start up to and including the first full stop or line break, plus following spaces.
- `REUTERS`: the word "reuters" anywhere, in any capitalisation (`re.IGNORECASE`).

scripts/transforms.py · Lines 109–114

```
IMAGE_CREDIT = re.compile(r"(featured\s+)?
(image|photo|pic|video|screenshot|screen\s+capture|audio|gif)s?"
                          r"\s+(via|credit|by|courtesy)\b[^\n]*", re.IGNORECASE)
URL = re.compile(r"https?://\S+|www\.\S+|pic\.twitter\.com/\S+", re.IGNORECASE)
HANDLE = re.compile(r"@\w+")
MEDIA_TAG = re.compile(r"[\[(](videos?|tweets?|images?|photos?|watch|audio)[\])]"
                       r"|\b(watch|breaking|read more|just in)\s*:", re.IGNORECASE)
```

Patterns for **E3b**, the fake sites' page furniture:

- `IMAGE_CREDIT`: optional "featured", one of image/photo/pic/video/screenshot/screen capture/audio/gif (optional plural `s?`), then via/credit/by/courtesy (`\b` = whole word), then the rest of the line (`[^\n]*`). Catches "Featured image via Justin Sullivan/Getty Images".
- `URL`: `http://` or `https://` or `www.` or `pic.twitter.com/` followed by non-space characters (`\S+`).
- `HANDLE`: `@` followed by word characters, e.g. "@realDonaldTrump".
- `MEDIA_TAG`: bracketed tags like "[Video]", "(Tweets)", "[Images]", or "watch:", "breaking:", "read more:", "just in:" (`|` means "or").

scripts/transforms.py · Lines 115–120

```
# The fake scrape lost apostrophes and quotation marks ("Trump s", "couldn t": a word
# followed by a lone s/t appears in 91.6% of fake vs 38.8% of real stripped articles),
# so every apostrophe and quote mark becomes a space in both classes. Ellipsis and dashes
# map to ASCII. (Extended once, after the E5c coefficients exposed the straight-quote case.)
TYPOGRAPHY = str.maketrans({"‘": " ", "’": " ", "'": " ", "“": " ", "”": " ", '"': " ",
                            "…": "...", "–": "-", "—": "-"})
```

`TYPOGRAPHY` is a translation table: each curly or straight apostrophe or quote mark → a space, "…" → "...", en dash and em dash → "-".

Why: the fake articles were scraped in a way that lost these characters ("tonight s", "couldn t"), while Reuters text kept them ("tonight's", curly quotes in 50.8% of real articles, 0% of fake). A model can separate the classes just by spotting an apostrophe. Turning every one into a space **in both classes** makes them look the same ("tonight s" in both).

Honesty point (from the comment): this rule was extended **once**, after the E5c coefficients showed the straight-quote case. It **lowered** our own headline numbers (E5c from 98.0% to 90.3%, E3c from 93.4% to 90.7%), and the before/after results are both kept (`results/before_quote_fix/`) and reported.

scripts/transforms.py · Lines 123–127

```
123  def strip_artifacts(text):
124      """E3: remove the dateline, the leading sentence and the word 'Reuters'."""
125      text = DATELINE.sub("", text, count=1)
126      text = LEADING_SENTENCE.sub("", text, count=1)
127      return REUTERS.sub("", text)
```

E3 on a body: remove the dateline once at the start (`count=1`), then the leading sentence (which in wire copy follows the dateline and is formulaic; E2a showed the opening line alone gives 99.1%), then every "reuters". Applied to **every** article, real and fake alike. These markers belong to the Reuters pipeline, so E3 removes one class's markers (RQ2's "from Reuters articles"), and E3b adds the other pipeline's.

scripts/transforms.py · Lines 130–134

```
130  def strip_page_furniture(text):
131      """E3b: remove credits, URLs, handles and media tags; normalise typography."""
132      for pattern in (IMAGE_CREDIT, URL, HANDLE, MEDIA_TAG):
133          text = pattern.sub(" ", text)
134      return text.translate(TYPOGRAPHY)
```

E3b: replace each furniture pattern with a **space** (not nothing, so the words on either side don't stick together), then apply the typography table.

scripts/transforms.py · Lines 137–139

```
137  def stripped_titles_bodies(df):
138      """(titles, bodies) after E3 stripping."""
139      return df["title"].map(lambda t: REUTERS.sub("", t)), df["text"].map(strip_artifacts)
```

E3 for a whole table: titles have no dateline or lead, so only "Reuters" is removed from them; bodies get the full `strip_artifacts` . `.map(f)` applies `f` to every row.

scripts/transforms.py · Lines 142–145

```
142  def fully_stripped_texts(df):
143      """Title + body after E3 and E3b stripping: the strictest text view."""
144      titles, bodies = stripped_titles_bodies(df)
145      return titles.map(strip_page_furniture) + " " + bodies.map(strip_page_furniture)
```

The strictest text view: E3 then E3b on titles and bodies, joined with a space. Used by E4b, E5 and the SVM check.

scripts/transforms.py · Lines 148–164

```
# ---------- probes on stripped text (function words + named entities, one spaCy pass) ----------

def stripped_profile(texts, cache_name="stripped_profile"):
    """For each (already stripped) text: its function-word string and its entity-label counts.

    cache_name must differ per dataset (ISOT uses the default) so caches don't overwrite each other.
    """
    def compute(texts):
        docs = _spacy_pipe(texts, ["parser", "lemmatizer"], "stripped-text profile")
        out = []
        for doc in docs:
            counts = {}
            for ent in doc.ents:
                counts[ent.label_] = counts.get(ent.label_, 0) + 1
            out.append({"fw": _function_word_string(doc), "ents": counts})
        return out
    return _cached(cache_name, texts, compute)
```

One spaCy pass on stripped text, with the entity recogniser **on**, gives two things per article: its function-word string (for E3c and E5c) and a count of each entity type (for E2e). Doing both in one pass saves a second ~25-minute run. `counts.get(label, 0) + 1` adds one to that type's count. `cache_name` must differ per dataset (ISOT uses `stripped_profile`, McIntire `mcintire_profile`), otherwise one dataset's cache would overwrite the other's.

scripts/transforms.py · Lines 167–172

```
def entity_features(profile, texts):
    """Named-entity counts per 100 words, one column per spaCy entity type."""
    counts = pd.DataFrame([p["ents"] for p in profile]).fillna(0)
    counts = counts[sorted(counts.columns)]
    words = pd.Series([max(len(t.split()), 1) for t in texts], index=counts.index)
    return counts.div(words, axis=0) * 100
```

E2e features. A DataFrame is built from the per-article count dictionaries; types an article doesn't mention become 0 (`fillna(0)`); columns are sorted so the order is always the same; each count is divided by the article's word count and × 100, i.e. **per 100 words**, so a long article doesn't score high just for being long. Only counts reach the model, never the names themselves.

scripts/transforms.py · Lines 175–199

```
175 # ---------- sentiment ----------
176
177 def sentiment_features(titles, bodies):
178     """VADER negative / neutral / positive / compound scores, for title and body separately."""
179     def compute(pairs):
180         import nltk
181         from nltk.sentiment import SentimentIntensityAnalyzer
182
183         nltk.download("vader_lexicon", quiet=True)
184         sia = SentimentIntensityAnalyzer()
185         print(f"sentiment: scoring {len(pairs)} documents with VADER (one-off)")
186         out = []
187         for i, pair in enumerate(pairs):
188             title, body = pair.split("\x01", 1)
189             t, b = sia.polarity_scores(title), sia.polarity_scores(body)
190             out.append([t["neg"], t["neu"], t["pos"], t["compound"],
191                         b["neg"], b["neu"], b["pos"], b["compound"]])
192             if (i + 1) % 5000 == 0:
193                 print(f"  {i + 1}/{len(pairs)}", flush=True)
194         return out
195
196     pairs = [f"{t}\x01{b}" for t, b in zip(titles, bodies)]
197     rows = _cached("sentiment", pairs, compute)
198     return pd.DataFrame(rows, columns=["title_neg", "title_neu", "title_pos", "title_compound",
199                                        "body_neg", "body_neu", "body_pos", "body_compound"])
```

E2d features. NLTK's VADER scorer gives four numbers for a piece of text: `neg`, `neu`, `pos` (shares of negative, neutral and positive wording, summing to 1) and `compound` (overall from −1, very negative, to +1, very positive). Title and body are scored separately → 8 columns.

Line 196 joins each title and body with `\x01` (a control character that never occurs in text) so the cache key covers both; line 188 splits them apart again. `nltk.download(..., quiet=True)` fetches VADER's word list the first time only.

7.4 scripts/00_get_data.py — fetch and verify data (step 0)

Purpose: make sure a reproducer has **exactly** the data the results came from.

scripts/00_get_data.py · Lines 1–7

```
"""Step 0 — fetch and verify the datasets.

LIAR and McIntire are downloaded automatically if missing. ISOT needs a Kaggle
account, so it is downloaded by hand (instructions are printed if it is missing).
Every file is then checked against the SHA-256 checksum of the copy the results
were produced from, so a reproducer knows they have exactly the same data.
"""
```

Docstring: LIAR and McIntire download automatically; ISOT needs a Kaggle login, so it can't.

scripts/00_get_data.py · Lines 9–15

```
import hashlib
import io
import sys
import urllib.request
import zipfile

from common import DATA_DIR
```

`hashlib` for SHA-256; `io` to treat downloaded bytes as a file; `sys` to exit with an error; `urllib.request` to download; `zipfile` to open zip archives.

scripts/00_get_data.py · Line 17

```
ISOT_URL = "https://www.kaggle.com/datasets/clmentbisaillon/fake-and-real-news-dataset"
```

Where ISOT can be downloaded, printed if it is missing.

scripts/00_get_data.py · Lines 19–29

```
# zip URL -> {member name inside the zip: destination path}
DOWNLOADS = {
    "https://sites.cs.ucsb.edu/~william/data/liar_dataset.zip": {
        "train.tsv": DATA_DIR / "liar" / "train.tsv",
        "valid.tsv": DATA_DIR / "liar" / "valid.tsv",
        "test.tsv": DATA_DIR / "liar" / "test.tsv",
    },

"https://raw.githubusercontent.com/joolsa/fake_real_news_dataset/master/fake_or_real_news.csv.zip": {
        "fake_or_real_news.csv": DATA_DIR / "mcintire" / "fake_or_real_news.csv",
    },
}
```

For each zip file URL, which files to take out of it and where to put them. Only the named files are extracted, nothing else in the archive.

scripts/00_get_data.py · Lines 31–39

```
31  SHA256 = {
32      DATA_DIR / "True.csv": "ba0844414a65dc6ae7402b8eee5306da24b6b56488d6767135af466c7dcb2775",
33      DATA_DIR / "Fake.csv": "bebf8bcfe95678bf2c732bf413a2ce5f621af0102c82bf08083b2e5d3c693d0c",
34      DATA_DIR / "liar" / "train.tsv":
"23ef8cb7ff57b25c941100d0744039cebc595d65a67ea190f72b330a658d8c36",
35      DATA_DIR / "liar" / "valid.tsv":
"8dd3d7ed1a11e78ed7df090c9eddb0f91147f895850c78d49dd7b4e88fd09a16",
36      DATA_DIR / "liar" / "test.tsv":
"ee41e2c741e0019216952625145af8c4aa8c8d3057ec68284b134267bb845e6f",
37      DATA_DIR / "mcintire" / "fake_or_real_news.csv":
38          "bb7fa746dd7148b63b0a10e47b329f4b4825afc85b206d3fea18ecfce28ee731",
39  }
```

The expected SHA-256 fingerprint of every data file. If even one byte differs (a re-uploaded dataset, a converted line ending), the fingerprint is completely different. This actually happened once: git's automatic line-ending conversion changed True.csv, the check caught it, and a `.gitattributes` rule (`data/** -text`) now stores data files byte for byte.

scripts/00_get_data.py · Lines 42–50

```
42  def download(url, members):
43      """Fetch a zip and write only the named members to their destinations."""
44      print(f"downloading {url}")
45      with urllib.request.urlopen(url, timeout=120) as response:
46          archive = zipfile.ZipFile(io.BytesIO(response.read()))
47      for member, dest in members.items():
48          dest.parent.mkdir(parents=True, exist_ok=True)
49          dest.write_bytes(archive.read(member))
50          print(f"  wrote {dest.relative_to(DATA_DIR.parent)}")
```

Downloads a zip (120-second timeout), keeps it in memory (`io.BytesIO`), creates the destination folder if needed and writes only the named files.

scripts/00_get_data.py · Lines 53–58

```
53  def sha256(path):
54      h = hashlib.sha256()
55      with open(path, "rb") as f:
56          for chunk in iter(lambda: f.read(1 << 20), b""):
57              h.update(chunk)
58      return h.hexdigest()
```

Computes a file's SHA-256 by reading it in 1 MB chunks (`1 << 20` = 2^{20} = 1,048,576 bytes), so large files never need to fit in memory. `iter(lambda: f.read(...), b"")` keeps reading until it gets empty bytes (end of file).

scripts/00_get_data.py · Lines 61–64

```
61  def main():
62      for url, members in DOWNLOADS.items():
63          if not all(dest.exists() for dest in members.values()):
64              download(url, members)
```

Downloads a dataset only if any of its files is missing.

scripts/00_get_data.py · Lines 66–76

```
66      problems = 0
67      for path, expected in SHA256.items():
68          name = path.relative_to(DATA_DIR.parent)
69          if not path.exists():
70              print(f"MISSING   {name}")
71              problems += 1
72          elif sha256(path) != expected:
73              print(f"DIFFERENT {name}  (not the file the results were produced from)")
74              problems += 1
75          else:
76              print(f"ok        {name}")
```

Checks every file and prints ok, MISSING or DIFFERENT, counting problems.

scripts/00_get_data.py · Lines 78–83

```
78      if not (DATA_DIR / "True.csv").exists() or not (DATA_DIR / "Fake.csv").exists():
79          print(f"\nISOT must be downloaded by hand (free Kaggle account needed):\n  {ISOT_URL}\n"
80                f"Unzip it and put True.csv and Fake.csv in {DATA_DIR}")
81      if problems:
82          sys.exit(f"\n{problems} data file(s) missing or different; fix before running the
experiments.")
83      print("\nall datasets present and verified")
```

Prints ISOT download instructions if needed. If anything is missing or different, `sys.exit` ends with an error code, so `run_all.py` stops before any experiment runs on the wrong data.

scripts/00_get_data.py · Lines 86–87

```
86  if __name__ == "__main__":
87      main()
```

Run `main()` when executed directly.

7.5 scripts/01_baseline.py — E1, the detector (step 1)

Purpose: build the standard detector and reproduce the familiar ISOT accuracy. This is the number the rest of the project interrogates.

scripts/01_baseline.py · Line 1

```
"""E1 — Baseline: TF-IDF + logistic regression on full text (title + body)."""
```

Docstring.

scripts/01_baseline.py · Line 3

```
from common import load_isot, print_metrics, run_experiment, save_top_tokens
```

Imports the four helpers from common.py.

scripts/01_baseline.py · Lines 6–8

```
def main():
    df = load_isot()
    texts = df["title"] + " " + df["text"]
```

Loads the cleaned data and makes each article's text: title + a space + body (the space stops the last word of the title from merging with the first word of the body).

scripts/01_baseline.py · Lines 10–12

```
    result = run_experiment(texts, df["label"], "e1_baseline")
    print_metrics(result)
    save_top_tokens(result)
```

Runs the experiment (TF-IDF + logistic regression, 5 splits), prints the metrics and saves the 30 strongest words per class.

scripts/01_baseline.py · Lines 15–16

```
if __name__ == "__main__":
    main()
```

Entry point.

Result: 98.4% accuracy (98.5% ± 0.2% over 5 splits), only 121 errors among 7,729 test articles (46 real called fake, 75 fake missed).

Strongest real words: reuters, said, on, washington, in, republican, wednesday, tuesday, thursday, presidential, told, had. **Strongest fake words:** via, video, this, us, just, image, hillary, is, gop, read, featured, that.

What to say: the strongest clues are about **who published** the article, not whether it is true: the agency name, the dateline city, the days of the week Reuters puts in every story, its attribution verbs ("said", "told"); and on the fake side, image-credit lines ("featured image via"), "video", "read" (from "READ MORE:") and an opinionated voice ("this", "just", "us"). Report Figure 3 shows these with the source markers in orange.

7.6 scripts/02_partial_input.py — E2a, E2b, E2c (step 2)

Purpose: the partial-input probes. Each model is denied the article's real content. If one still scores far above chance, the dataset leaks the label through what it *was* given.

scripts/02_partial_input.py · Lines 1–7

```
"""E2 — Partial-input probes: models deliberately denied the article's content.

E2a  first sentence of the body only
E2b  function words only (spaCy POS tags; cached to data/cache/)
E2c  eight metadata numbers only, random forest
plus a body-only run (no title) used by the degradation figure.
"""
```

Docstring listing the conditions.

scripts/02_partial_input.py · Lines 9–12

```
from sklearn.ensemble import RandomForestClassifier

from common import RANDOM_STATE, load_isot, print_metrics, run_experiment, save_top_tokens
from transforms import WHITESPACE_TFIDF, first_sentence, function_words, metadata_features
```

Imports the random forest, the helpers and the views from transforms.py.

scripts/02_partial_input.py · Lines 15–18

```
def main():
    df = load_isot()
    labels = df["label"]
    full = (df["title"] + " " + df["text"]).tolist()
```

Loads data; `full` is title + body as a plain Python list (spaCy and the cache work on lists).

scripts/02_partial_input.py · Lines 20–23

```
    print("=== E2a first sentence only ===")
    result = run_experiment(df["text"].map(first_sentence), labels, "e2a_first_sentence")
    print_metrics(result)
    save_top_tokens(result, k=15)
```

E2a, first sentence only. `df["text"].map(first_sentence)` keeps only the body up to the first full stop. 15 strongest words per class are saved.

Result: 99.1%, *higher* than the full article. The real side's top words are the dateline: reuters, washington, said, fla, calif, wednesday, london. The opening line is almost a pure source label; the rest of the article only dilutes it.

scripts/02_partial_input.py · Lines 25–29

```
    print("\n=== E2b function words only ===")
    result = run_experiment(function_words(full), labels, "e2b_function_words",
                            tfidf_kwargs=WHITESPACE_TFIDF)
    print_metrics(result)
    save_top_tokens(result, k=15)
```

E2b, function words only. Every article becomes its function-word string, e.g. "the of a and to , the -". No nouns, verbs, names or numbers survive. `WHITESPACE_TFIDF` keeps one-character tokens.

Result: 96.8%. The headline finding. Top fake tokens: ":", "via", "?", "!", "this", "like", "that", "you", "why". Top real tokens: "...", "on", "-", "'s" and curly quote characters. The two classes are told apart by

punctuation habits and grammatical voice.

scripts/02_partial_input.py · Lines 31–40

```
31      print("\n=== E2c metadata only ===")
32      features = metadata_features(full)
33      forest = RandomForestClassifier(random_state=RANDOM_STATE, n_jobs=-1)
34      result = run_experiment(features, labels, "e2c_metadata", model=forest)
35      print_metrics(result)
36      fitted = result["pipeline"].named_steps["clf"]  # run_experiment fits a copy, not `forest`
itself
37      importances = sorted(zip(features.columns, fitted.feature_importances_), key=lambda p: -p[1])
38      print("  feature importances:")
39      for feature, importance in importances:
40          print(f"    {importance:.3f}  {feature}")
```

E2c, eight numbers only, with a random forest (`random_state` fixed so the forest is identical each run; `n_jobs=-1` = use all cores).

Line 36 matters: `run_experiment` trains a **clone**, not the `forest` object itself, so the importances must be read from the returned pipeline's `clf` step. Line 37 pairs each column name with its importance and sorts from largest (`-p[1]` sorts descending).

Result: 86.2% from eight numbers. Most important: question marks (0.187), average word length (0.175), share of all-caps words (0.132), exclamation marks (0.120). Fake articles ask more questions, shout more and use shorter words.

scripts/02_partial_input.py · Lines 42–44

```
42      print("\n=== body only (no title), for the degradation figure ===")
43      result = run_experiment(df["text"], labels, "e2_body_only")
44      print_metrics(result)
```

Full body without the title, needed for the degradation chart (report Figure 8): 98.4%.

scripts/02_partial_input.py · Lines 47–48

```
47  if __name__ == "__main__":
48      main()
```

Entry point.

7.7 scripts/03_stripped.py — E3, E3b (step 3)

Purpose: the obvious objection is "just delete the word Reuters". This tests it: remove every identifiable source marker and retrain on the full remaining text.

scripts/03_stripped.py · Lines 1–21

```
"""E3 — Artifact removal: strip source markers, retrain on full text (rules in transforms.py).

E3  (as planned) applied identically to real and fake articles:
  1. the dateline, e.g. 'WASHINGTON (Reuters) - '
  2. the leading sentence of the body
  3. the word 'Reuters' anywhere (title or body, any case)
E3b additionally strips the fake sites' page furniture, i.e. text that is not
article prose, and normalises typography:
  4. media credits ('Featured image via ... Getty Images', 'Video via YouTube',
     'Screenshot via ...'), to the end of the line
  5. URLs and pic.twitter.com links
  6. Twitter @handles
  7. media tags: '[Video]', '(Tweets)', 'WATCH:', 'BREAKING:', 'READ MORE:'
  8. every apostrophe and quotation mark (curly or straight) replaced by a space,
     ellipsis and dashes mapped to ASCII: the fake scrape lost these characters
     (no curly quotes at all in fake vs 50.8% of real; "Trump s", "couldn t")

The E1 -> E3 / E3b gap measures how much of the signal lives in removable
markers (result: very little). The top coefficients printed afterwards show
what is still separable once these are gone.
"""
```

The docstring lists all eight removal rules (implemented in transforms.py lines 101–145). Rules 1–3 are E3 (the Reuters pipeline's markers), 4–8 are added in E3b (the fake sites' page furniture and the typography). The last paragraph states what the comparison measures.

scripts/03_stripped.py · Lines 23–24

```
from common import load_isot, print_metrics, run_experiment, save_top_tokens
from transforms import stripped_titles_bodies, strip_page_furniture
```

Imports.

scripts/03_stripped.py · Lines 27–29

```
def main():
    df = load_isot()
    titles, bodies = stripped_titles_bodies(df)
```

Loads data and applies E3 to titles and bodies.

scripts/03_stripped.py · Lines 31–34

```
    print("=== E3 source markers stripped ===")
    result = run_experiment(titles + " " + bodies, df["label"], "e3_stripped")
    print_metrics(result)
    save_top_tokens(result)
```

E3: 97.7% (from 98.4%). Removing the dateline, the first sentence and every "Reuters" costs under one point.

scripts/03_stripped.py · Lines 36–40

```
36      print("\n=== E3b source markers + page furniture stripped ===")
37      texts = titles.map(strip_page_furniture) + " " + bodies.map(strip_page_furniture)
38      result = run_experiment(texts, df["label"], "e3b_furniture_stripped")
39      print_metrics(result)
40      save_top_tokens(result)
```

E3b: additionally strips the furniture from both titles and bodies. **97.3%.** Errors rise only from 121 to 206.

Strongest real words afterwards: said, on, told, in, republican, says, reporters, had, spokesman, nov: wire-service attribution. Strongest fake: via, this, us, hillary, gop, just, that, is, here, mr: an opinionated blog voice. "via" survives in prose such as "via Raw Story" (crediting other sites). **What to say:** the source is not in a few removable words; it runs through the whole way the article is written.

scripts/03_stripped.py · Lines 43–44

```
43  if __name__ == "__main__":
44      main()
```

Entry point.

7.8 scripts/04_cross_corpus.py — E4, E4b (step 4)

Purpose: the decisive test of whether the detector learned "fake news" or "ISOT": train on ISOT, test on datasets it has never seen, labelled independently.

scripts/04_cross_corpus.py · Lines 1–19

```
"""E4 — Cross-corpus transfer.

E4 (headlines, LIAR). LIAR has no article bodies, so the comparison is headline-level:
  e4_isot_titles      in-domain control: ISOT titles only, usual 80/20 split
  e4_liar_transfer    train on all ISOT titles, test on all 12,836 LIAR statements
  e4_liar_no_halftrue the same without the ambiguous half-true label
The control shows how much the drop is due to crossing datasets rather than to
headlines simply carrying less information than full articles.

E4b (full articles, McIntire "fake or real news", 6,335 articles). Its real
articles come from many mainstream outlets (not Reuters), so a model that learned
"Reuters style = real" should fail on it:
  e4b_mcintire_control         in-domain control: McIntire alone, usual split
  e4b_mcintire_full            train ISOT full text, test McIntire full text
  e4b_mcintire_stripped        both sides stripped with the E3 + E3b rules
  e4b_mcintire_function_words  stripped function words on both sides
  e4b_reverse                  train McIntire, test ISOT (full text)
Source: github.com/joolsa/fake_real_news_dataset (fake_or_real_news.csv).
"""
```

Docstring with all eight runs. Why two external datasets: LIAR has **no article bodies** (only short statements), so E4 can only compare headlines. McIntire has **full articles** whose real side comes from many outlets, not Reuters, so E4b tests the full-text detector itself.

scripts/04_cross_corpus.py · Lines 21–26

```
import csv

import pandas as pd

from common import DATA_DIR, FAKE, REAL, load_isot, print_metrics, run_experiment
from transforms import WHITESPACE_TFIDF, fully_stripped_texts, stripped_profile
```

`csv` is needed for `QUOTE_NONE` (below). The other imports are the usual helpers and views.

scripts/04_cross_corpus.py · Lines 28–33

```
# The TSVs have no header row; column names are from Wang (2017).
LIAR_COLUMNS = [
    "id", "label", "statement", "subject", "speaker", "job", "state", "party",
    "barely_true_counts", "false_counts", "half_true_counts", "mostly_true_counts",
    "pants_on_fire_counts", "context",
]
```

LIAR's files have no header row; these 14 column names come from the LIAR paper (Wang, 2017). Only `label` and `statement` are used.

scripts/04_cross_corpus.py · Lines 34–38

```
# Six-way labels collapsed to binary: the three "true" grades are real, the rest fake.
LIAR_BINARY = {
    "true": REAL, "mostly-true": REAL, "half-true": REAL,
    "barely-true": FAKE, "false": FAKE, "pants-fire": FAKE,
}
```

LIAR rates statements on six levels. We collapse them to two: true, mostly-true and half-true count as **real**; barely-true, false and pants-on-fire ("pants on fire" = blatantly false) as **fake**. Half-true is the judgement call, so line 77 reruns without it.

scripts/04_cross_corpus.py · Lines 41–49

```
41  def load_liar():
42      parts = [
43          pd.read_csv(DATA_DIR / "liar" / f"{split}.tsv", sep="\t", header=None,
44                      names=LIAR_COLUMNS, quoting=csv.QUOTE_NONE)
45          for split in ("train", "valid", "test")
46      ]
47      liar = pd.concat(parts, ignore_index=True)
48      liar["binary"] = liar["label"].map(LIAR_BINARY)
49      return liar
```

Reads the three LIAR files (train, valid, test). We don't train on LIAR, so all 12,836 statements are used as one test set. `quoting=csv.QUOTE_NONE` tells pandas that quote marks are ordinary characters: LIAR statements contain unbalanced quotes that would otherwise make rows merge. Line 48 adds the binary label.

scripts/04_cross_corpus.py · Lines 52–58

```
52  def load_mcintire():
53      """McIntire corpus, cleaned like ISOT: empty bodies and duplicate bodies dropped."""
54      df = pd.read_csv(DATA_DIR / "mcintire" / "fake_or_real_news.csv")
55      df[["title", "text"]] = df[["title", "text"]].fillna("")
56      df = df[df["text"].str.strip() != ""].drop_duplicates(subset="text", keep="first")
57      df["label"] = df["label"].map({"REAL": REAL, "FAKE": FAKE})
58      return df.reset_index(drop=True)
```

McIntire, cleaned exactly like ISOT (empty and duplicate bodies dropped): 6,335 → 6,059 articles (2,989 real, 3,070 fake). Its text labels "REAL"/"FAKE" become 0/1.

scripts/04_cross_corpus.py · Lines 61–64

```
61  def main():
62      isot = load_isot()
63      liar = load_liar()
64      print(f"LIAR: {len(liar)} statements, six-way labels
{liar['label'].value_counts().to_dict()}")
```

Loads both datasets and prints LIAR's six-way label counts.

scripts/04_cross_corpus.py · Lines 66–68

```
66      print("\n=== E4 control: ISOT titles only, in-domain ===")
67      result = run_experiment(isot["title"], isot["label"], "e4_isot_titles")
68      print_metrics(result)
```

E4 control: ISOT headlines only, normal split. **94.6%**. So headlines alone are enough within ISOT; any drop on LIAR is due to changing dataset, not to headlines being too short.

scripts/04_cross_corpus.py · Lines 70–73

```
70      print("\n=== E4 transfer: train ISOT titles, test LIAR ===")
71      result = run_experiment(isot["title"], isot["label"], "e4_liar_transfer",
72                              test_features=liar["statement"], test_labels=liar["binary"])
73      print_metrics(result)
```

E4 transfer: train on all ISOT headlines, test on all LIAR statements. **45.5%**, **below** chance (55.8%). It called 5,575 of the 7,167 "real" statements fake (78%). Explanation: it learned "not written like Reuters = fake",

and politicians' statements aren't written like Reuters.

scripts/04_cross_corpus.py · Lines 75–80

```
75      # Sensitivity check on the label mapping: half-true is the judgement call, so drop it.
76      print("\n=== E4 transfer, LIAR without half-true ===")
77      clear = liar[liar["label"] != "half-true"]
78      result = run_experiment(isot["title"], isot["label"], "e4_liar_no_halftrue",
79                              test_features=clear["statement"], test_labels=clear["binary"])
80      print_metrics(result)
```

Sensitivity check: without half-true: **51.1%** vs 55.6% chance. Still at or below chance, so the result doesn't depend on our label mapping.

scripts/04_cross_corpus.py · Lines 82–86

```
82      mc = load_mcintire()
83      print(f"\nMcIntire: {len(mc)} articles after cleaning, "
84            f"{(mc['label'] == REAL).sum()} real / {(mc['label'] == FAKE).sum()} fake")
85      isot_full, mc_full = isot["title"] + " " + isot["text"], mc["title"] + " " + mc["text"]
86      isot_stripped, mc_stripped = fully_stripped_texts(isot), fully_stripped_texts(mc)
```

Loads McIntire and prepares the four text versions: full text (title + body) and fully stripped text (E3 + E3b rules) for both datasets.

scripts/04_cross_corpus.py · Lines 88–97

```
88      runs = [
89          ("e4b_mcintire_control", mc_full, mc["label"], None, None, None),
90          ("e4b_mcintire_full", isot_full, isot["label"], mc_full, mc["label"], None),
91          ("e4b_mcintire_stripped", isot_stripped, isot["label"], mc_stripped, mc["label"], None),
92          ("e4b_mcintire_function_words",
93           [p["fw"] for p in stripped_profile(isot_stripped.tolist())], isot["label"],
94           [p["fw"] for p in stripped_profile(mc_stripped.tolist(),
cache_name="mcintire_profile")],
95            mc["label"], WHITESPACE_TFIDF),
96          ("e4b_reverse", mc_full, mc["label"], isot_full, isot["label"], None),
97      ]
```

The five E4b runs as a list of (name, training text, training labels, test text, test labels, TF-IDF settings). `None` as test text means "split normally" (the control). The function-word run reads them from the stripped-text spaCy cache for each dataset (`mcintire_profile` for McIntire).

scripts/04_cross_corpus.py · Lines 98–102

```
98      for name, train_x, train_y, test_x, test_y, tfidf_kwargs in runs:
99          print(f"\n=== {name} ===")
100         result = run_experiment(train_x, train_y, name, tfidf_kwargs=tfidf_kwargs,
101                                 test_features=test_x, test_labels=test_y)
102         print_metrics(result)
```

Runs each. **Results:** McIntire in-domain control **91.9%** (the task is learnable on McIntire itself); ISOT → McIntire full text **61.2%**, stripped 59.8%, function words 63.9% (chance 50.7%); reverse, McIntire → ISOT, 65.0%. The ISOT detector calls 2,022 of McIntire's 2,989 real articles fake (68%).

scripts/04_cross_corpus.py · Lines 105–106

```
105 if __name__ == "__main__":
106     main()
```

Entry point.

7.9 scripts/05_more_probes.py — E2d, E2e, E3c, SVM check (step 5)

Purpose: more probes, all on **fully stripped** text, so they cannot use any marker already found. Anything they find is new. Plus the classifier check.

scripts/05_more_probes.py · Lines 1–11

```
"""More probes, all on the fully stripped text (E3 + E3b rules), plus a model check.

The new probes run on stripped text so they cannot use any marker already
identified (Reuters, datelines, credits, curly quotes). Anything they find is new.

E2d  sentiment only: eight VADER scores (title and body), random forest
E2e  named entities only: counts of each spaCy entity type per 100 words, random forest
E3c  function words only, taken from the stripped text
SVM  E1, E2a, E2b and E3b repeated with a linear SVM instead of logistic regression,
     to show the results do not depend on the choice of classifier
"""
```

Docstring.

scripts/05_more_probes.py · Lines 13–19

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import LinearSVC

from common import RANDOM_STATE, load_isot, print_metrics, run_experiment, save_top_tokens
from transforms import (WHITESPACE_TFIDF, entity_features, first_sentence, fully_stripped_texts,
                        function_words, sentiment_features, strip_page_furniture,
                        stripped_profile, stripped_titles_bodies)
```

Imports, including `LinearSVC` for the SVM check.

scripts/05_more_probes.py · Lines 22–30

```
def forest_with_importances(features, labels, name):
    forest = RandomForestClassifier(random_state=RANDOM_STATE, n_jobs=-1)
    result = run_experiment(features, labels, name, model=forest)
    print_metrics(result)
    fitted = result["pipeline"].named_steps["clf"]  # run_experiment fits a copy, not `forest`
itself
    importances = sorted(zip(features.columns, fitted.feature_importances_), key=lambda p: -p[1])
    print("  feature importances (top 8):")
    for feature, importance in importances[:8]:
        print(f"    {importance:.3f}  {feature}")
```

A small helper so E2d and E2e don't repeat code: train a random forest on a numeric table, print the metrics and the eight most important features (read from the fitted clone, as in step 2).

scripts/05_more_probes.py · Lines 33–38

```
def main():
    df = load_isot()
    labels = df["label"]
    titles, bodies = stripped_titles_bodies(df)
    titles, bodies = titles.map(strip_page_furniture), bodies.map(strip_page_furniture)
    stripped = (titles + " " + bodies).tolist()
```

Builds the fully stripped title and body (E3 then E3b) once, and the joined text as a list.

scripts/05_more_probes.py · Lines 40–41

```
40      print("=== E2d sentiment only (stripped text) ===")
41      forest_with_importances(sentiment_features(titles.tolist(), bodies.tolist()), labels,
"e2d_sentiment")
```

E2d, sentiment only: 8 VADER scores per article. **73.8%.** Most important: title and body compound scores. Sentiment alone is a weaker but real signal: fake articles are more emotional.

scripts/05_more_probes.py · Line 43

```
43      profile = stripped_profile(stripped)
```

One spaCy pass (cached) gives function words and entity counts for every stripped article.

scripts/05_more_probes.py · Lines 45–46

```
45      print("\n=== E2e named entities only (stripped text) ===")
46      forest_with_importances(entity_features(profile, stripped), labels, "e2e_entities")
```

E2e, named entities only: counts per 100 words of each entity type. **81.1%.** Most important: GPE (places), DATE, NORP (groups). Wire stories are full of places and dates.

scripts/05_more_probes.py · Lines 48–52

```
48      print("\n=== E3c function words only, from stripped text ===")
49      result = run_experiment([p["fw"] for p in profile], labels, "e3c_stripped_function_words",
50                              tfidf_kwargs=WHITESPACE_TFIDF)
51      print_metrics(result)
52      save_top_tokens(result, k=15)
```

E3c, function words of the stripped text. No content words and none of the removed markers. **90.7%.** Top fake: "!", ":", "?", "like", "this", "that", "why", "you" (exclamation, rhetorical questions, direct address). Top real: "on", "--", "had", "would", "could", "whether", "amid" (reported, hedged speech).

scripts/05_more_probes.py · Lines 54–65

```
54      print("\n=== Model check: linear SVM ===")
55      full = (df["title"] + " " + df["text"]).tolist()
56      svm_inputs = [
57          ("e1_svm", full, None),
58          ("e2a_svm", df["text"].map(first_sentence), None),
59          ("e2b_svm", function_words(full), WHITESPACE_TFIDF),
60          ("e3b_svm", fully_stripped_texts(df), None),
61      ]
62      for name, texts, tfidf_kwargs in svm_inputs:
63          result = run_experiment(texts, labels, name, model=LinearSVC(random_state=RANDOM_STATE),
64                                  tfidf_kwargs=tfidf_kwargs)
65          print_metrics(result)
```

SVM check: E1, E2a, E2b and E3b rerun with `LinearSVC` instead of logistic regression. Results: E1 99.3%, E2a 99.7%, E2b 97.5%, E3b 98.1%: slightly **higher** every time. A different (and slightly stronger) classifier exploits the shortcut more, not less.

scripts/05_more_probes.py · Lines 68–69

```
68  if __name__ == "__main__":
69      main()
```

Entry point.

7.10 scripts/06_confounds.py — E5, topic-matched (step 6)

Purpose: rule out topic. Real articles are 53% US politics, 47% world news; fake articles are all US/political news. So "world news = real" could explain part of the results.

scripts/06_confounds.py · Lines 1–15

```
"""E5 — Topic-matched confound check: is the signal style, or topic?

ISOT's real articles are 53% US politics ('politicsNews') and 47% world news
('worldnews'); its fake articles are all US politics and US news. So part of any
result could be "world news = real". E5 keeps only US-politics real articles and
all fake articles, then repeats the strictest conditions:

E5a  full text
E5b  fully stripped text (E3 + E3b rules)
E5c  function words only, from the stripped text
E5d  eight metadata numbers, from the stripped text

The subject field is used only to choose rows, never as model input. Chance
changes on this subset (fake becomes the majority class).
"""
```

Docstring. Note line 13: the `subject` column is used only to **choose rows**, never as model input (it would be a leak itself, since the two classes use different subject names).

scripts/06_confounds.py · Lines 17–20

```
from sklearn.ensemble import RandomForestClassifier

from common import FAKE, RANDOM_STATE, load_isot, print_metrics, run_experiment, save_top_tokens
from transforms import WHITESPACE_TFIDF, fully_stripped_texts, metadata_features,
stripped_profile
```

Imports.

scripts/06_confounds.py · Lines 23–26

```
def main():
    df = load_isot()
    stripped = fully_stripped_texts(df)
    profile = stripped_profile(stripped.tolist())  # cached by step 5
```

Loads data; builds the fully stripped text and its spaCy profile for all articles (from cache).

scripts/06_confounds.py · Lines 28–32

```
    keep = ((df["label"] == FAKE) | (df["subject"] == "politicsNews")).to_numpy()
    sub = df[keep]
    labels = sub["label"]
    print(f"topic-matched subset: {len(sub)} articles, "
          f"{(labels != FAKE).sum()} real (politicsNews) / {(labels == FAKE).sum()} fake")
```

`keep` is True for every fake article and for real articles whose subject is `politicsNews`. All world-news articles are removed. Result: **28,666 articles, 11,213 real, 17,453 fake.** Fake is now the majority, so **chance rises to 60.9%**. `.to_numpy()` turns it into a plain True/False array usable on both the table and the stripped texts.

scripts/06_confounds.py · Lines 34–36

```
34      print("\n=== E5a topic-matched, full text ===")
35      result = run_experiment(sub["title"] + " " + sub["text"], labels, "e5a_topic_full")
36      print_metrics(result)
```

E5a, full text: 98.3%. Topic matching changes almost nothing.

scripts/06_confounds.py · Lines 38–41

```
38      print("\n=== E5b topic-matched, fully stripped ===")
39      result = run_experiment(stripped[keep], labels, "e5b_topic_stripped")
40      print_metrics(result)
41      save_top_tokens(result, k=15)
```

E5b, fully stripped: 96.7%.

scripts/06_confounds.py · Lines 43–48

```
43      print("\n=== E5c topic-matched, stripped, function words only ===")
44      function_words = [p["fw"] for p, k in zip(profile, keep) if k]
45      result = run_experiment(function_words, labels, "e5c_topic_function_words",
46                              tfidf_kwargs=WHITESPACE_TFIDF)
47      print_metrics(result)
48      save_top_tokens(result, k=15)
```

E5c, stripped function words: 90.3% vs 60.9% chance. The strictest test in the project: same topic, no markers, no content words, and still ~30 points above chance. `zip(profile, keep)` keeps the cached function-word strings of the selected rows. (This is the result that went from 98.0% to 90.3% when the quote rule was extended.)

scripts/06_confounds.py · Lines 50–53

```
50      print("\n=== E5d topic-matched, stripped, metadata only ===")
51      forest = RandomForestClassifier(random_state=RANDOM_STATE, n_jobs=-1)
52      result = run_experiment(metadata_features(stripped[keep]), labels, "e5d_topic_metadata",
model=forest)
53      print_metrics(result)
```

E5d, eight numbers from stripped text: 85.8%.

scripts/06_confounds.py · Lines 56–57

```
56  if __name__ == "__main__":
57      main()
```

Entry point.

7.11 scripts/07_figures.py — figures and results table (step 7)

Purpose: draw every chart and the results table from the saved JSON files. It **never retrains** anything, so figures can be restyled in seconds without changing a single number.

scripts/07_figures.py · Lines 1–11

```
"""Figures and results table. Reads results/*.json only; never retrains anything.

figures/fig1_accuracy.png      accuracy across all conditions, chance line
figures/fig2_leakage_tokens.png top 15 E1 tokens per class, provenance markers highlighted
figures/fig3_confusion.png     E1 vs E3 vs E3b confusion matrices
figures/fig4_degradation.png   accuracy as information is removed
figures/fig5_topic_matched.png E5: the strictest probes on US-politics-only data
figures/fig6_svm_check.png     logistic regression vs linear SVM on the same inputs
figures/fig7_cross_corpus.png  within-dataset vs other-dataset accuracy (LIAR, McIntire)
results/results_table.md       the results table for the report
"""
```

Docstring listing every output file.

scripts/07_figures.py · Lines 13–20

```
import json

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.colors import LinearSegmentedColormap
from matplotlib.ticker import PercentFormatter

from common import RESULTS_DIR, ROOT
```

`matplotlib.pyplot` draws charts; `LinearSegmentedColormap` makes the blue shading for the confusion matrices; `PercentFormatter` shows axes as percentages.

scripts/07_figures.py · Lines 22–28

```
FIGURES_DIR = ROOT / "figures"

# Validated palette (dataviz reference instance, light mode).
BLUE, ORANGE = "#2a78d6", "#eb6834"
INK, INK_2, MUTED = "#0b0b0b", "#52514e", "#898781"
GRID, AXIS, SURFACE = "#e1e0d9", "#c3c2b7", "#fcfcfb"
NEUTRAL_BAR = "#b5b4ad"
```

Output folder and colours: blue for the main data, orange to highlight (source markers, the SVM), greys for text, grid lines and background. Two colours only, chosen to stay distinguishable for colour-blind readers.

scripts/07_figures.py · Lines 30–36

```
# Tokens that identify the publishing pipeline rather than the claim: agency name,
# dateline, timestamp format, image credits, URL / embed fragments, media tags.
# Hand-labelled; deliberately strict (attribution verbs like 'said' are left out).
PROVENANCE = {
    "reuters", "washington", "edt", "via", "image", "images", "getty", "featured",
    "pic", "https", "com", "www", "twitter", "video", "watch", "read", "breaking", "wire",
}
```

`PROVENANCE`: words we label by hand as **publisher markers** (agency, dateline, timestamp, image credit, URL fragments, media tags). Figure 2 colours them orange. Deliberately strict: "said" is left out even though it is a wire-style word, so we don't overstate.

scripts/07_figures.py · Lines 38–68

```
CONDITIONS = [  # (results file, label for figures and table, what the model sees); figure 1 rows
    ("e1_baseline", "E1  Full text", "Title + body"),
    ("e2a_first_sentence", "E2a  First sentence only", "Opening line of the body"),
    ("e2b_function_words", "E2b  Function words only", "Grammatical words + punctuation"),
    ("e2c_metadata", "E2c  Metadata only", "Eight numbers, no text"),
    ("e2d_sentiment", "E2d  Sentiment only", "Eight VADER scores, stripped text"),
    ("e2e_entities", "E2e  Named entities only", "Entity-type counts, stripped text"),
    ("e3_stripped", "E3  Source markers stripped", "Full text minus dateline, 'Reuters', lead"),
    ("e3b_furniture_stripped", "E3b  + page furniture stripped",
     "E3 minus credits, URLs, handles, media tags; plain quotes"),
    ("e3c_stripped_function_words", "E3c  Stripped, function words only", "E3b text reduced to
function words"),
    ("e4_isot_titles", "E4 control  ISOT headlines", "Headline only, same dataset"),
    ("e4_liar_transfer", "E4  ISOT → LIAR", "Headline-trained, tested on LIAR"),
]
TABLE_ONLY = [  # extra rows for the results table (different test sets or chance levels)
    ("e4_liar_no_halftrue", "E4  ISOT → LIAR, no half-true", "LIAR without the ambiguous label"),
    ("e4b_mcintire_control", "E4b control  McIntire in-domain", "McIntire full text, its own
split"),
    ("e4b_mcintire_full", "E4b  ISOT → McIntire, full text", "Full articles, other dataset"),
    ("e4b_mcintire_stripped", "E4b  ISOT → McIntire, stripped", "Both sides E3b-stripped"),
    ("e4b_mcintire_function_words", "E4b  ISOT → McIntire, function words", "Stripped function
words"),
    ("e4b_reverse", "E4b  McIntire → ISOT, full text", "Reverse direction"),
    ("e5a_topic_full", "E5a  US politics only: full text", "Topic-matched subset"),
    ("e5b_topic_stripped", "E5b  US politics only: stripped", "Topic-matched, E3b text"),
    ("e5c_topic_function_words", "E5c  US politics only: stripped function words",
     "Topic-matched, no markers, no content words"),
    ("e5d_topic_metadata", "E5d  US politics only: stripped metadata", "Topic-matched, eight
numbers"),
    ("e1_svm", "E1  Full text (SVM)", "As E1, linear SVM"),
    ("e2a_svm", "E2a  First sentence (SVM)", "As E2a, linear SVM"),
    ("e2b_svm", "E2b  Function words (SVM)", "As E2b, linear SVM"),
    ("e3b_svm", "E3b  Page furniture stripped (SVM)", "As E3b, linear SVM"),
]
```

Every condition: (results file, label, what the model sees). `CONDITIONS` are drawn in the accuracy chart; `TABLE_ONLY` appear only in the table, because they have a different chance level or test set and would mislead on the same axis.

scripts/07_figures.py · Lines 70–84

```
plt.rcParams.update({
    "font.family": ["Segoe UI", "DejaVu Sans"],
    "font.size": 10,
    "text.color": INK,
    "axes.labelcolor": INK_2,
    "axes.edgecolor": AXIS,
    "axes.facecolor": SURFACE,
    "figure.facecolor": SURFACE,
    "xtick.color": MUTED,
    "ytick.color": INK_2,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "savefig.dpi": 200,
    "savefig.bbox": "tight",
})
```

Global chart style: font, colours, no top/right border lines, 200 dpi (dots per inch) so figures are sharp in print, and `bbox: tight` to trim empty margins.

scripts/07_figures.py · Lines 87–97

```
def load(name):
    path = RESULTS_DIR / f"{name}.json"
    if not path.exists():
        return None
    with open(path, encoding="utf-8") as f:
        return json.load(f)


def title(ax, main, sub):
    ax.set_title(main, loc="left", fontsize=12, fontweight="semibold", pad=22)
    ax.text(0, 1.02, sub, transform=ax.transAxes, fontsize=9, color=INK_2, va="bottom")
```

`load` reads one results file (or returns `None` if that experiment hasn't been run, so the figure is skipped instead of crashing). `title` puts a bold title and a grey subtitle above a chart.

scripts/07_figures.py · Lines 100–125

```
def fig1_accuracy(results):
    rows = [(label, results[name]) for name, label, _ in CONDITIONS if results.get(name)]
    labels = [label for label, _ in rows][::-1]
    acc = [r["accuracy"] for _, r in rows][::-1]
    chance = results["e1_baseline"]["majority_baseline"]

    fig, ax = plt.subplots(figsize=(8, 0.5 * len(rows) + 1.4))
    y = np.arange(len(rows))
    ax.barh(y, acc, height=0.55, color=BLUE, zorder=2)
    for yi, a in zip(y, acc):
        ax.text(a + 0.01, yi, f"{a:.1%}", va="center", fontsize=9, color=INK)
    ax.axvline(chance, color=INK_2, linestyle=(0, (4, 3)), linewidth=1, zorder=3)
    ax.text(chance, -0.75, f" chance {chance:.1%}", fontsize=8.5, color=INK_2, va="center")
    ax.set_ylim(-1, len(rows) - 0.5)

    ax.set_yticks(y, labels)
    ax.set_xlim(0, 1.08)
    ax.xaxis.set_major_formatter(PercentFormatter(1.0, decimals=0))
    ax.grid(axis="x", color=GRID, linewidth=0.8, zorder=0)
    ax.tick_params(axis="y", length=0)
    title(ax, "Accuracy by condition",
          "Chance = always guessing the majority class on the ISOT test split "
          "(LIAR's is %.1f%%)" % (100 * results["e4_liar_transfer"]["majority_baseline"])
          if results.get("e4_liar_transfer") else "Chance = majority class on the test split")
    fig.savefig(FIGURES_DIR / "fig1_accuracy.png")
    plt.close(fig)
```

`fig1_accuracy.png` (report Figure 2), accuracy by condition. Horizontal bars (`barh`), one per condition, with the percentage written at the end of each bar and a dashed vertical line at chance. Lists are reversed (`[::-1]`) so E1 appears at the top.

scripts/07_figures.py · Lines 128–150

```
def fig2_tokens(tokens):
    fig, axes = plt.subplots(1, 2, figsize=(10, 5.2))
    for ax, cls, heading in ((axes[0], "real", "Predicts REAL"), (axes[1], "fake", "Predicts
FAKE")):
        top = tokens[cls][:15][::-1]
        words = [w for w, _ in top]
        weights = [abs(c) for _, c in top]
        colors = [ORANGE if w in PROVENANCE else NEUTRAL_BAR for w in words]
        y = np.arange(len(top))
        ax.barh(y, weights, height=0.6, color=colors, zorder=2)
        ax.set_yticks(y, words)
        ax.tick_params(axis="y", length=0)
        ax.grid(axis="x", color=GRID, linewidth=0.8, zorder=0)
        ax.set_xlabel("|coefficient|")
        ax.set_title(heading, loc="left", fontsize=11, fontweight="semibold")

    handles = [plt.Rectangle((0, 0), 1, 1, color=ORANGE), plt.Rectangle((0, 0), 1, 1,
color=NEUTRAL_BAR)]
    fig.legend(handles, ["Source / page-furniture marker", "Other token"], loc="upper left",
               bbox_to_anchor=(0.01, 1.0), ncol=2, frameon=False, fontsize=9)
    fig.suptitle("Strongest E1 logistic regression tokens per class", x=0.01, y=1.06,
                 ha="left", fontsize=12, fontweight="semibold")
    fig.tight_layout()
    fig.savefig(FIGURES_DIR / "fig2_leakage_tokens.png")
    plt.close(fig)
```

`fig2_leakage_tokens.png` **(report Figure 3), strongest E1 words.** Two panels (real, fake), 15 words each, bar length = size of the weight. Words in `PROVENANCE` are orange, the rest grey, with a legend.

scripts/07_figures.py · Lines 153–181

```
def fig3_confusion(panels):
    """panels: list of (heading, result) pairs, drawn side by side on one colour scale."""
    cmap = LinearSegmentedColormap.from_list("blue", ["#f0f5fc", "#cde2fb", "#6da7ec", "#2a78d6",
"#104281"])
    matrices = [np.array(r["confusion_matrix"]) for _, r in panels]
    vmax = max(m.max() for m in matrices)

    fig, axes = plt.subplots(1, len(panels), figsize=(4.3 * len(panels), 3.9))
    for ax, m, (heading, r) in zip(axes, matrices, panels):
        ax.imshow(m, cmap=cmap, vmin=0, vmax=vmax)
        row_totals = m.sum(axis=1, keepdims=True)
        for i in range(2):
            for j in range(2):
                text_color = "white" if m[i, j] > vmax * 0.5 else INK
                ax.text(j, i, f"{m[i, j]:,}\n{m[i, j] / row_totals[i, 0]:.1%}", ha="center",
                        va="center", color=text_color, fontsize=10)
        ax.set_xticks([0, 1], ["Real", "Fake"])
        ax.set_yticks([0, 1], ["Real", "Fake"])
        ax.set_xlabel("Predicted")
        ax.set_ylabel("Actual")
        ax.tick_params(length=0)
        for spine in ax.spines.values():
            spine.set_visible(False)
        ax.set_title(f"{heading}\naccuracy {r['accuracy']:.1%}, {m[0, 1] + m[1, 0]} errors",
                     loc="left", fontsize=10.5)
    fig.suptitle("Confusion matrices, same test split (row % in each cell)", x=0.01, ha="left",
                 fontsize=12, fontweight="semibold")
    fig.tight_layout()
    fig.savefig(FIGURES_DIR / "fig3_confusion.png")
    plt.close(fig)
```

`fig3_confusion.png` **(report Figure 4), confusion matrices** for E1, E3 and E3b side by side, all on **one colour scale** (`vmax` = largest cell) so the shading is comparable. Each cell shows the count and the row percentage; text turns white on dark cells. The panel title gives accuracy and total errors.

scripts/07_figures.py · Lines 184–209

```
184 def fig4_degradation(results):
185     steps = [("e1_baseline", "Full text"), ("e2_body_only", "Body only"),
186              ("e2b_function_words", "Function words"), ("e2c_metadata", "Metadata only")]
187     steps = [(label, results[name]["accuracy"]) for name, label in steps if results.get(name)]
188     chance = results["e1_baseline"]["majority_baseline"]
189     x = np.arange(len(steps))
190     acc = [a for _, a in steps]
191 
192     fig, ax = plt.subplots(figsize=(7.5, 4))
193     ax.plot(x, acc, color=BLUE, linewidth=2, solid_capstyle="round", zorder=2)
194     ax.scatter(x, acc, s=70, color=BLUE, edgecolor=SURFACE, linewidth=2, zorder=3)
195     for xi, a in zip(x, acc):
196         ax.text(xi, a + 0.025, f"{a:.1%}", ha="center", fontsize=9)
197     ax.axhline(chance, color=INK_2, linestyle=(0, (4, 3)), linewidth=1)
198     ax.text(-0.25, chance + 0.01, f"chance {chance:.1%}", va="bottom", fontsize=8.5, color=INK_2)
199 
200     ax.set_xticks(x, [label for label, _ in steps])
201     ax.set_xlim(-0.3, len(steps) - 0.4)
202     ax.set_ylim(0.4, 1.05)
203     ax.yaxis.set_major_formatter(PercentFormatter(1.0, decimals=0))
204     ax.grid(axis="y", color=GRID, linewidth=0.8, zorder=0)
205     ax.tick_params(axis="x", length=0)
206     ax.set_xlabel("Less information available to the model →")
207     title(ax, "Accuracy as content is removed", "Logistic regression throughout, except metadata
(random forest)")
208     fig.savefig(FIGURES_DIR / "fig4_degradation.png")
209     plt.close(fig)
```

`fig4_degradation.png` **(report Figure 8), degradation.** Accuracy as information is removed: full text → body only → function words → eight numbers, with the chance line. The point: accuracy falls slowly and stays far above chance.

scripts/07_figures.py · Lines 212–240

```
212 def fig5_topic_matched(results):
213     steps = [("e5a_topic_full", "Full text"), ("e5b_topic_stripped", "All markers stripped"),
214              ("e5c_topic_function_words", "Stripped, function words only"),
215              ("e5d_topic_metadata", "Stripped, eight numbers only")]
216     steps = [(label, results[name]) for name, label in steps if results.get(name)]
217     if not steps:
218         return
219     labels = [label for label, _ in steps][::-1]
220     acc = [r["accuracy"] for _, r in steps][::-1]
221     chance = steps[0][1]["majority_baseline"]
222 
223     fig, ax = plt.subplots(figsize=(8, 0.5 * len(steps) + 1.6))
224     y = np.arange(len(steps))
225     ax.barh(y, acc, height=0.55, color=BLUE, zorder=2)
226     for yi, a in zip(y, acc):
227         ax.text(a + 0.01, yi, f"{a:.1%}", va="center", fontsize=9, color=INK)
228     ax.axvline(chance, color=INK_2, linestyle=(0, (4, 3)), linewidth=1, zorder=3)
229     ax.text(chance, -0.75, f" chance {chance:.1%}", fontsize=8.5, color=INK_2, va="center")
230     ax.set_ylim(-1, len(steps) - 0.5)
231     ax.set_yticks(y, labels)
232     ax.set_xlim(0, 1.08)
233     ax.xaxis.set_major_formatter(PercentFormatter(1.0, decimals=0))
234     ax.grid(axis="x", color=GRID, linewidth=0.8, zorder=0)
235     ax.tick_params(axis="y", length=0)
236     title(ax, "Topic-matched check: US politics only, both classes",
237           "Real world-news articles removed, so topic cannot separate the classes. "
238           "Chance = majority class (fake) on this subset")
239     fig.savefig(FIGURES_DIR / "fig5_topic_matched.png")
240     plt.close(fig)
```

`fig5_topic_matched.png` **(report Figure 6), topic-matched check (E5a–E5d)** with this subset's own chance line (60.9%).

scripts/07_figures.py · Lines 243–274

```
243  def fig6_svm_check(results):
244      pairs = [("e1_baseline", "e1_svm", "E1  Full text"), ("e2a_first_sentence", "e2a_svm", "E2a
First sentence"),
245               ("e2b_function_words", "e2b_svm", "E2b  Function words"),
246               ("e3b_furniture_stripped", "e3b_svm", "E3b  Page furniture stripped")]
247      pairs = [(results[a]["accuracy"], results[b]["accuracy"], label)
248               for a, b, label in pairs if results.get(a) and results.get(b)]
249      if not pairs:
250          return
251      pairs = pairs[::-1]
252      y = np.arange(len(pairs))
253
254      fig, ax = plt.subplots(figsize=(8, 0.55 * len(pairs) + 1.7))
255      for yi, (lr, svm, _) in zip(y, pairs):
256          ax.plot([lr, svm], [yi, yi], color=GRID, linewidth=2, zorder=1)
257      ax.scatter([p[0] for p in pairs], y, s=70, color=BLUE, edgecolor=SURFACE, linewidth=2,
zorder=3,
258                 label="Logistic regression")
259      ax.scatter([p[1] for p in pairs], y, s=70, color=ORANGE, edgecolor=SURFACE, linewidth=2,
zorder=3,
260                 label="Linear SVM")
261      for yi, (lr, svm, _) in zip(y, pairs):
262          ax.text(min(lr, svm) - 0.002, yi, f"{lr:.1%} / {svm:.1%}  ", va="center", ha="right",
263                  fontsize=8.5, color=INK_2)
264      ax.set_yticks(y, [p[2] for p in pairs])
265      lo = min(min(p[0], p[1]) for p in pairs)
266      ax.set_xlim(np.floor(lo * 50) / 50 - 0.02, 1.0)
267      ax.xaxis.set_major_formatter(PercentFormatter(1.0, decimals=0))
268      ax.grid(axis="x", color=GRID, linewidth=0.8, zorder=0)
269      ax.tick_params(axis="y", length=0)
270      ax.legend(loc="upper left", bbox_to_anchor=(0, -0.18), ncol=2, frameon=False, fontsize=9)
271      title(ax, "Same inputs, different classifier", "Accuracy with logistic regression vs a linear
SVM "
272            "(axis starts above zero; points, not bars)")
273      fig.savefig(FIGURES_DIR / "fig6_svm_check.png")
274      plt.close(fig)
```

`fig6_svm_check.png` **(report Figure 7), classifier check.** A dot plot: blue = logistic regression, orange = SVM, joined by a grey line, for the same four inputs. The axis doesn't start at zero, so dots are used instead of bars (bars starting above zero would exaggerate differences).

scripts/07_figures.py · Lines 277–310

```
def fig7_cross_corpus(results):
    groups = [  # (label, within-dataset result, across-dataset result)
        ("ISOT headlines → LIAR", "e4_isot_titles", "e4_liar_transfer"),
        ("ISOT articles → McIntire", "e1_baseline", "e4b_mcintire_full"),
        ("McIntire articles → ISOT", "e4b_mcintire_control", "e4b_reverse"),
    ]
    groups = [(label, results[a], results[b]) for label, a, b in groups if results.get(a) and
results.get(b)]
    if not groups:
        return
    groups = groups[::-1]
    y = np.arange(len(groups))
    h = 0.32

    fig, ax = plt.subplots(figsize=(8, 0.95 * len(groups) + 1.6))
    ax.barh(y + h / 2 + 0.02, [g[1]["accuracy"] for g in groups], height=h, color=BLUE, zorder=2,
            label="Tested on the same dataset")
    ax.barh(y - h / 2 - 0.02, [g[2]["accuracy"] for g in groups], height=h, color=ORANGE,
zorder=2,
            label="Tested on the other dataset")
    for yi, (_, within, across) in zip(y, groups):
        for off, r in ((h / 2 + 0.02, within), (-h / 2 - 0.02, across)):
            ax.text(r["accuracy"] + 0.01, yi + off, f"{r['accuracy']:.1%}", va="center",
fontsize=8.5, color=INK)
        chance = across["majority_baseline"]
        ax.plot([chance, chance], [yi - h - 0.08, yi + 0.02], color=INK_2, linestyle=(0, (3, 2)),
                linewidth=1, zorder=3)
    ax.set_yticks(y, [g[0] for g in groups])
    ax.set_xlim(0, 1.08)
    ax.xaxis.set_major_formatter(PercentFormatter(1.0, decimals=0))
    ax.grid(axis="x", color=GRID, linewidth=0.8, zorder=0)
    ax.tick_params(axis="y", length=0)
    ax.legend(loc="upper left", bbox_to_anchor=(0, -0.12), ncol=2, frameon=False, fontsize=9)
    title(ax, "Accuracy collapses on a different dataset",
          "Dashed tick = chance (majority class) on the dataset tested")
    fig.savefig(FIGURES_DIR / "fig7_cross_corpus.png")
    plt.close(fig)
```

`fig7_cross_corpus.png` (report Figure 5), cross-corpus. For each direction, a blue bar (tested on the same dataset) and an orange bar (tested on the other dataset), with a dashed tick at the other dataset's chance.

scripts/07_figures.py · Lines 313–326

```
def results_table(results):
    lines = ["| Condition | What the model sees | Accuracy | Mean ± SD, 5 splits | F1 (fake) |
Chance |",
             "|---|---|---|---|---|---|"]
    for name, label, sees in CONDITIONS + TABLE_ONLY:
        r = results.get(name)
        if r:
            spread = (f"{r['accuracy_mean']:.1%} ± {r['accuracy_sd']:.1%}"
                      if "accuracy_mean" in r else "n/a (fixed test set)")
            lines.append(f"| {label} | {sees} | {r['accuracy']:.1%} | {spread} | {r['f1']:.3f} |
"
                         f"{r['majority_baseline']:.1%} |")
    table = "\n".join(lines) + "\n"
    with open(RESULTS_DIR / "results_table.md", "w", encoding="utf-8") as f:
        f.write(table)
    print(table)
```

Writes `results/results_table.md` (the table in section 8): accuracy, mean ± SD (or "n/a" for fixed external test sets), F1 and chance for every condition.

scripts/07_figures.py · Lines 329–350

```
def main():
    FIGURES_DIR.mkdir(exist_ok=True)
    names = {name for name, _, _ in CONDITIONS + TABLE_ONLY} | {"e2_body_only"}
    results = {name: load(name) for name in names}
    missing = sorted(n for n, r in results.items() if r is None)
    if missing:
        print(f"missing results (run their scripts first): {', '.join(missing)}")

    fig1_accuracy(results)
    tokens = load("e1_baseline_tokens")
    if tokens:
        fig2_tokens(tokens)
    panels = [("E1  Full text", "e1_baseline"), ("E3  Source markers stripped", "e3_stripped"),
              ("E3b  + page furniture stripped", "e3b_furniture_stripped")]
    panels = [(heading, results[name]) for heading, name in panels if results.get(name)]
    fig3_confusion(panels)
    fig4_degradation(results)
    fig5_topic_matched(results)
    fig6_svm_check(results)
    fig7_cross_corpus(results)
    results_table(results)
    print(f"figures written to {FIGURES_DIR}")
```

`main` loads every results file, lists any missing, and draws each figure and the table.

scripts/07_figures.py · Lines 353–354

```
if __name__ == "__main__":
    main()
```

Entry point.

8. Results: what every number means

The full table, exactly as produced by step 7. "Accuracy" is the seed-42 split; "Mean ± SD" is over five random splits; "Chance" is always guessing the larger class of that test set.

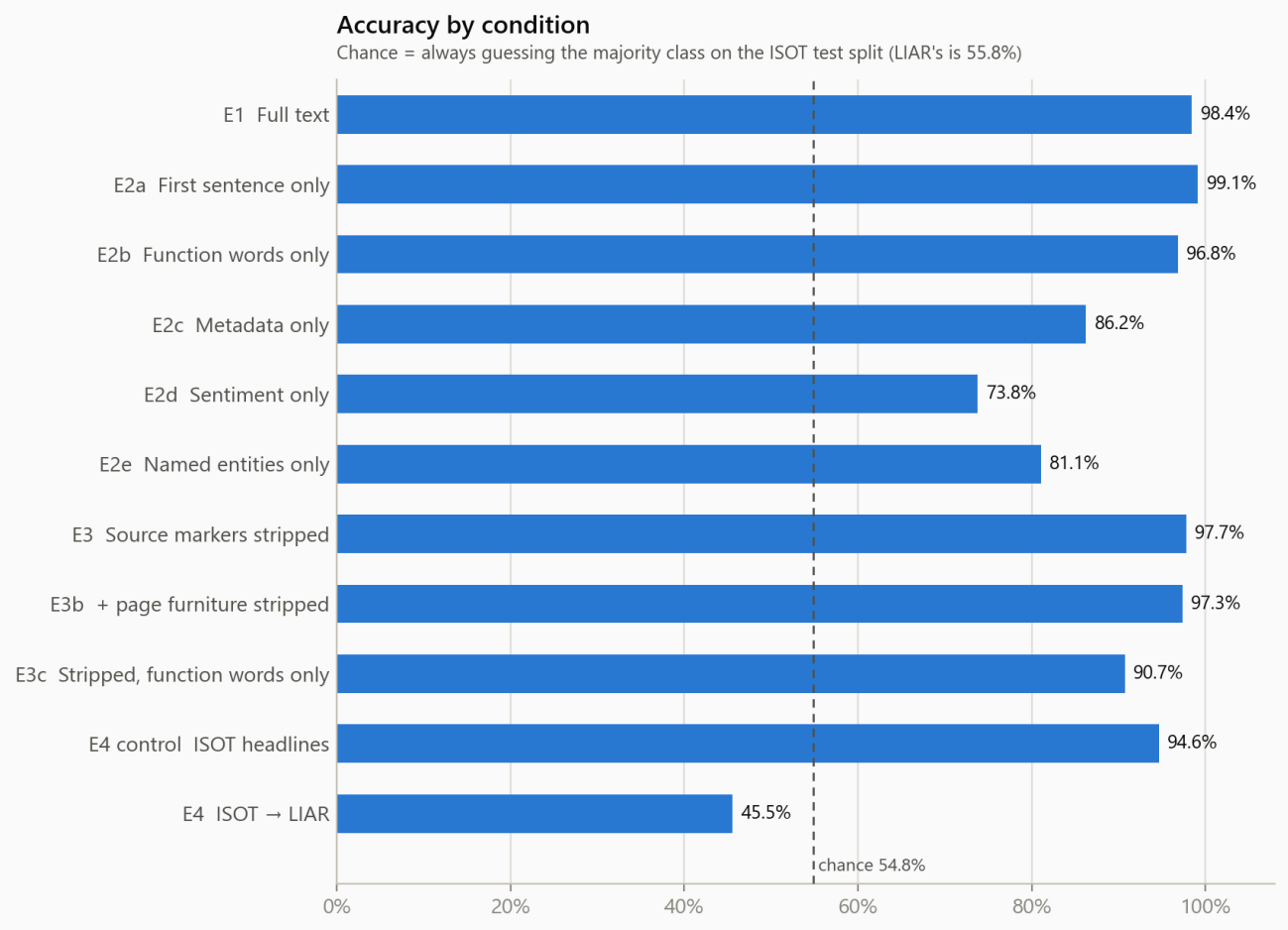
Condition	What the model sees	Accuracy	Mean ± SD	F1 (fake)	Chance
E1 Full text	Title + body	98.4%	98.5% ± 0.2%	0.983	54.8%
E2a First sentence only	Opening line of the body	99.1%	99.0% ± 0.1%	0.990	54.8%
E2b Function words only	Grammatical words + punctuation	96.8%	96.8% ± 0.2%	0.964	54.8%
E2c Metadata only	Eight numbers, no text	86.2%	86.3% ± 0.1%	0.842	54.8%
E2d Sentiment only	Eight VADER scores, stripped text	73.8%	74.1% ± 0.5%	0.696	54.8%
E2e Named entities only	Entity-type counts, stripped text	81.1%	81.6% ± 0.4%	0.782	54.8%
E3 Source markers stripped	Full text minus dateline, 'Reuters', lead	97.7%	97.7% ± 0.2%	0.975	54.8%
E3b + page furniture stripped	E3 minus credits, URLs, handles, media tags; plain quotes	97.3%	97.3% ± 0.1%	0.970	54.8%
E3c Stripped, function words only	E3b text reduced to function words	90.7%	90.8% ± 0.1%	0.895	54.8%
E4 control ISOT headlines	Headline only, same dataset	94.6%	94.7% ± 0.4%	0.939	54.8%
E4 ISOT → LIAR	Headline-trained, tested on LIAR	45.5%	n/a (fixed test set)	0.549	55.8%
E4 ISOT → LIAR, no half-true	LIAR without the ambiguous label	51.1%	n/a (fixed test set)	0.631	55.6%
E4b control McIntire in-domain	McIntire full text, its own split	91.9%	91.2% ± 0.5%	0.921	50.7%
E4b ISOT → McIntire, full text	Full articles, other dataset	61.2%	n/a (fixed test set)	0.700	50.7%
E4b ISOT → McIntire, stripped	Both sides E3b-stripped	59.8%	n/a (fixed test set)	0.687	50.7%
E4b ISOT → McIntire, function words	Stripped function words	63.9%	n/a (fixed test set)	0.671	50.7%
E4b McIntire → ISOT, full text	Reverse direction	65.0%	n/a (fixed test set)	0.648	54.8%
E5a US politics only: full text	Topic-matched subset	98.3%	98.5% ± 0.1%	0.986	60.9%
E5b US politics only: stripped	Topic-matched, E3b text	96.7%	97.0% ± 0.2%	0.973	60.9%
E5c US politics only: stripped function words	Topic-matched, no markers, no content words	90.3%	90.5% ± 0.2%	0.920	60.9%
E5d US politics only: stripped metadata	Topic-matched, eight numbers	85.8%	85.6% ± 0.3%	0.884	60.9%
E1 Full text (SVM)	As E1, linear SVM	99.3%	99.3% ± 0.1%	0.992	54.8%
E2a First sentence (SVM)	As E2a, linear SVM	99.7%	99.6% ± 0.1%	0.996	54.8%
E2b Function words (SVM)	As E2b, linear SVM	97.5%	97.4% ± 0.1%	0.972	54.8%
E3b Page furniture stripped (SVM)	As E3b, linear SVM	98.1%	98.1% ± 0.1%	0.979	54.8%

How to read it, group by group:

- **E1 vs E2 (RQ1).** A model that sees only the first sentence (99.1%), only function words (96.8%) or only eight numbers (86.2%) does almost as well as, or better than, the full-text detector. Content is not what the detector uses.

- **E3, E3b, E3c (RQ2).** Removing every source marker from both classes costs about one point (98.4 → 97.3%). Even then, function words alone give 90.7%. The source is in the writing style.
- **E4, E4b (RQ3).** Within its own data the detector is excellent; on other data it is at or below chance on LIAR and 61.2% on McIntire, calling two-thirds of genuine articles fake. McIntire's own model gets 91.9% on McIntire, so the task there is learnable; the ISOT model just didn't learn it.
- **E5 (RQ4, topic).** On US politics only, stripped, function words: 90.3% vs 60.9% chance. Topic does not explain the signal.
- **SVM rows (RQ4, classifier).** A different classifier does slightly better on every condition.
- **F1** moves with accuracy everywhere except the cross-corpus rows, where it can stay moderate while accuracy is low, because the model predicts "fake" for most articles (high recall, poor precision).

Limitations to state openly: the probes were run in depth on ISOT only (LIAR and McIntire are used as test sets); only classical models (no transformer such as BERT); the stripping rules are hand-written and were extended once (disclosed, with before/after numbers).



9. Presenter brief

What this section is. A *presenter handoff* (or brief) is a short package that lets someone present work they did not build: the story in order, the numbers to memorise, what to say about each figure, and how to answer questions. This section is that brief.

The story, in order (about 3 minutes)

Figure numbers are the report's.

1. **Hook.** "Fake news detectors on the ISOT benchmark report 98 to 99 percent. We built one and got 98.4. Then we tested whether that number means anything."
2. **The flaw.** "Every real article in ISOT comes from Reuters; every fake one from a set of flagged websites. So a model could be recognising the newsroom instead of the lie." Hospital analogy.
3. **The method.** "Partial-input baselines, from Gururangan et al. 2018: we train models that are *not allowed* to see the content. If they still score high, the dataset is leaking the answer."
4. **Probes (Figure 2).** "The first sentence alone: 99.1. Only words like 'the, of, a' and punctuation: 96.8. Eight numbers like word count and question marks: 86.2. Chance is 54.8."
5. **Why (Figure 3).** "The strongest clues are 'Reuters', 'Washington', days of the week, 'featured image via'. Publisher markers, not truth."
6. **Stripping (Figure 4).** "We removed every marker from both classes. Still 97.3."
7. **Topic (Figure 6).** "Same topic only, everything stripped, function words only: 90.3 against 60.9 chance."
8. **Other datasets (Figure 5).** "Trained on ISOT, tested elsewhere: 45.5 on LIAR, below chance; 61.2 on a dataset of real articles from other outlets, calling two-thirds of genuine news fake."
9. **Conclusion.** "The 98 percent measures the newsroom, not the truth. Fake news detectors should be tested across sources and alongside partial-input baselines, not on a single benchmark split."

Numbers to memorise

Number	What it is
98.4%	E1, our detector, full text
54.8%	chance on ISOT
99.1%	E2a, first sentence only
96.8%	E2b, function words only (the headline)
86.2%	E2c, eight numbers, no words
97.3%	E3b, every marker stripped from both classes
90.3% vs 60.9%	E5c, US politics only + stripped + function words, vs chance
45.5% vs 55.8%	E4, ISOT → LIAR, below chance
61.2% vs 50.7%	E4b, ISOT → McIntire full articles; 68% of real articles called fake
38,644	articles after cleaning (from 44,898; 631 empty, 5,623 duplicates removed)

Figure talking points (figure numbers as in the report)

Figure	Say
1 Architecture	Every condition goes through one function with the same split and classifier; only the input changes.
2 Accuracy by condition	Every bar, even the ones with no content, is far right of the chance line.
3 Strongest tokens	Orange bars are publisher markers. The detector's best clues are about who wrote it.
4 Confusion matrices	Stripping markers barely changes the errors: 121 → 206 of 7,729.
5 Cross-corpus	Blue = same dataset, orange = other dataset. The orange bars fall to chance.
6 Topic-matched	Same topic, still far above its 60.9% chance.
7 SVM check	Orange dots (SVM) are slightly right of blue: a better model exploits the shortcut more.
8 Degradation	Remove more and more information; accuracy falls slowly and never near chance.

Suggested split: the presenter tells the story and walks the figures; the code owner takes questions about implementation (sections 4 and 7). Agree beforehand who answers "why logistic regression", "why stratified" and "did you change anything after seeing results" (both should know the answers).

Framing rules. It *is* a fake news detector: we built one, and it is the thing under test. Never claim the project is the first to find problems with ISOT (Zhou et al. 2021 and Verma 2026 found related problems); our contribution is the style-only probes, stripping both classes, the topic control and two cross-corpus tests. State limitations without being asked.

10. Viva questions and answers

What's your accuracy?
Our detector gets 98.4%, like published results. The project shows that number doesn't measure fake news detection: models that can't see the content score almost the same, and the detector fails on other datasets.

So is this a fake news detector or not?
Yes: we built one (E1), and then tested it. The detector is the object under test. The finding is about what its accuracy means.

Isn't fake news detection a common, solved project?
It looks solved because ISOT makes it look easy. We show why: the classes can be told apart by publisher style alone.

Hasn't someone already shown this?
Partly. Zhou et al. (2021) showed fake news datasets leak source identity, and Verma (2026, a preprint) audited this same corpus by removing tags and metadata. Our stripping and transfer results agree with theirs. What we add: style-only probes (function words: 96.8%), stripping both classes' markers, a topic-matched control and a full-article cross-corpus test.

What is a partial-input baseline?
A model deliberately given only part of the input. If it still scores well, that part leaks the label. Gururangan et al. (2018) used it to expose artifacts in NLI data.

Why logistic regression and not BERT?
Its weights are readable: one per word, so we can see exactly what it relies on (report Figure 3). It's the standard TF-IDF baseline and matches published ISOT results. A stronger model would likely exploit the same shortcuts at least as well (the SVM already does) and would hide what it uses. Transformers are named as a limitation and as future work.

Why a random forest for E2c/E2d/E2e?
Those inputs are a few numbers, where useful rules are thresholds and combinations; trees capture those. It also gives feature importances.

Why stratified splitting?
It keeps the real/fake ratio identical in train and test, so accuracy and chance aren't shifted by a lucky split, and chance is the same across experiments.

Why 80/20 and seed 42?
80/20 is conventional and leaves 7,729 test articles (≈77 per percentage point). The seed fixes the split so every experiment uses the same test articles and anyone gets identical numbers. Any fixed number works; 42 is convention.

How do you know results aren't a lucky split?
Each experiment is repeated on 5 random splits; SDs are 0.1–0.5 points, much smaller than the differences we interpret.

Why is chance 54.8% and not 50%?
After removing empty and duplicate articles (mostly fake), ISOT is 54.8% real. We measure chance on each test set rather than assume 50%.

Why remove duplicates and empty articles?

Duplicates across train and test let the model be tested on articles it memorised. 630 of 631 empty bodies were fake, so 'empty = fake' would be a leak we created.

Why does the first sentence beat the full article?

In Reuters copy the opening contains the dateline (city, '(Reuters)'). It's an almost pure source label; the rest of the article dilutes it.

Why did removing 'Reuters' barely change anything?

The source isn't in one word; it's the whole register: wire reporting says 'said', 'told reporters', 'amid'; partisan blogs say 'this', 'just', 'you', '!' and '?'.

Isn't it just topic? Real has world news.

Tested in E5: US politics only, all markers stripped, function words only: 90.3% vs 60.9% chance.

Is it because of your classifier?

No. A linear SVM is slightly higher on every condition we reran.

Why is LIAR below chance?

The model learned 'not written like Reuters = fake'. Politicians' statements aren't written like Reuters, so it calls 78% of the true ones fake.

LIAR is only short statements. Unfair?

That's why there's an in-domain control (ISOT headlines: 94.6%) and a second, full-article test (McIntire: 61.2%, while a model trained on McIntire gets 91.9%).

Why count half-true as real?

It's a judgement call, so we also ran it without half-true: 51.1% vs 55.6% chance. Same conclusion.

Did you change anything after seeing results?

Once, disclosed: E5c's coefficients showed straight apostrophes still separated the classes (a scraping artifact), so the typography rule was extended to all quote marks in both classes. It lowered our own numbers (E5c 98.0 → 90.3%). Both versions are kept and reported.

Why no stemming or lemmatisation?

They merge word forms (said/says → say). The signal we study is in function words and punctuation, which stemming doesn't change, and the published ISOT baselines don't stem. We do use POS tagging, NER, sentiment, regex cleaning and TF-IDF.

Why not remove stop words?

Stop words are function words: the style signal lives there. Removing them would hide the effect we are measuring.

Could TF-IDF leak test information?

No. It sits inside a scikit-learn Pipeline, so its vocabulary and IDF weights are learned from the training split only.

What would fix the benchmark?

Evaluate across sources (train on some publishers, test on others), report partial-input baselines next to the main result, and build datasets where real and fake come from the same kinds of sources, labelled per claim.

Can I reproduce it?

Yes: one command, about 5 minutes, with checksum-verified data and fixed seeds; the README lists the exact numbers you should get.

What are the limitations?

Probes in depth on ISOT only; classical models only; hand-written stripping rules (extended once, disclosed). The topic, classifier, label-mapping and full-article objections were tested, not just mentioned.